



OULUN YLIOPISTO
UNIVERSITY of OULU

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Jarmo Vainio

THE USE OF DATA VISUALIZATION IN FUZZ TEST MONITORING

Master's Thesis
Degree Programme in Computer Science and Engineering
April 2014

Vainio J. (2014) The Use of Data Visualization in Fuzz Test Monitoring.
University of Oulu, Department of Computer Science and Engineering. Master's Thesis, 44 p.

ABSTRACT

Security vulnerabilities are a serious concern in today's software development. Complex software applications and systems may contain flaws that can be exploited by a remote attacker to execute malicious code. One method of finding vulnerabilities and bugs is by using a method of robustness testing, fuzzing. Fuzzers are mostly automated tools that are used to find flaws and unexpected behavior in the system under test.

Nowadays it is not uncommon to have multiple fuzzers running simultaneously on different systems in order to find bugs more efficiently. Although the fuzz testing process is mostly automated, there can be value in providing information about the testing process and its results in a visual form.

This work studies the use of data visualization techniques in monitoring fuzz testing processes. This is accomplished by developing a network based monitoring system that collects and visualizes test data. The monitoring system is used to run test scenarios and simulations to study the usefulness of data visualization, and its different techniques in fuzz testing.

The results from the performed case studies show that two-dimensional visualization techniques can effectively provide useful information about the fuzz testing process. The results also indicate that interactive visualization techniques are especially useful for testing process monitoring scenarios.

Key words: security vulnerability, two-dimensional, graphics

Vainio J. (2013) Datan visualisoinnin hyödyntäminen fuzz-testauksen valvonnassa. Oulun yliopisto, Tietotekniikan osasto. Diplomityö, 44 s.

TIIVISTELMÄ

Tärkeä osa nykypäivän ohjelmistokehitystä on mahdollisten tietoturva-aukkojen löytäminen ja paikkaaminen. Etenkin monimutkaiset ohjelmistot ja järjestelmät voivat sisältää virheitä, joita ulkopuolinen taho voi käyttää haitallisen toiminnan ajamiseen tietokoneella. Yksi keino tietoturva-aukkojen löytämiseen on fuzz-testauksen käyttäminen. Fuzzerit ovat enimmäkseen automatisoituja ohjelmia, joita käytetään löytämään tilanteita, joissa ohjelmisto kaatuu tai käyttäytyy odottamattomasti. Nykyään on yleistä käyttää useita fuzzereita rinnakkain eri järjestelmissä, jotta ohjelmistovirheitä löydettäisiin tehokkaammin. Fuzz-testauksen valvonta, varsinkin useiden järjestelmien tilanteissa, voi hyötyä testausprosessin tiedon esittämisestä visuaalisin keinoin.

Tässä työssä tutkitaan datan visualisoinnin tekniikoita fuzz-testauksen valvonnan yhteydessä. Työssä kehitetään verkkopohjainen valvontajärjestelmä, joka kerää ja visualisoi testidataa muilta järjestelmiltä. Valvontajärjestelmää käytetään visualisoimaan dataa sekä yksittäisestä fuzzerista, että laajemmista monen fuzzerin simulaatioista.

Saatujen tulosten perusteella nähdään, että kaksiulotteisten visualisointitekniikoiden avulla voidaan saada hyödyllistä ja helposti ymmärrettävää tietoa fuzzausprosessista. Tulosten mukaan erityisen hyödyllisiksi fuzzauksen valvonnassa osottautuivat visualisaatiot, jotka tarjoavat interaktiivisuutta käyttäjälle.

Avainsanat: tietoturva-aukko, kaksiulotteinen, grafiikka

TABLE OF CONTENTS

ABSTRACT

TIIVISTELMÄ

TABLE OF CONTENTS

FOREWORD

LIST OF ABBREVIATIONS AND SYMBOLS

1.	INTRODUCTION	7
2.	FUZZING	9
2.1.	Phases of Fuzzing	9
2.2.	Types of Fuzzers	10
2.3.	Fuzzing Target Monitoring	11
2.3.1.	Valid Case Instrumentation	11
2.3.2.	System Monitoring	12
2.3.3.	Monitoring With a Debugger	12
2.4.	State of the Art	12
3.	DATA VISUALIZATION	14
3.1.	Uses of Visualization	15
3.1.1.	Explanatory Visualization	16
3.1.2.	Exploratory Visualization	16
3.2.	Recent Developments	16
3.3.	Visualization Methods for System Monitoring	18
4.	DESIGN AND IMPLEMENTATION	21
4.1.	The Monitoring Server	22
4.1.1.	Web Frameworks	22
4.1.2.	Visualization Frameworks	22
4.2.	The Remote Host	24
4.3.	Overall System Architecture	26
5.	CASE STUDIES	27
5.1.	Case 1: Single Host in an Idle State	27
5.2.	Case 2: Single Host under Test	29
5.3.	Case 3: Multiple Host Visualizations	31
5.3.1.	Circle-packing Layout	32
5.3.2.	Multiple Horizon Graphs	33
5.3.3.	Large Multivariate Datasets	34
6.	DISCUSSION	36
6.1.	Requirements Revisited	36
6.2.	Possible Future Developments	38
7.	SUMMARY	40
8.	REFERENCES	41

FOREWORD

This thesis was done in the Oulu University Secure Programming Group (OUSPG) research group. The idea for this thesis evolved from my period of internship at OUSPG, during which I worked with process monitoring and fuzzers.

I would like to express my gratitude to Professor Juha Rönning for supervising this work and for providing valuable comments and criticism. I would also like to give special thanks to my technical supervisor, Christian Wieser, for providing advice and support during the writing process. I would also like to give my thanks to everyone at OUSPG for their support, and especially to Aki Helin and Atte Kettunen for their helpful technical insights.

Oulu, April 3rd, 2014

Jarmo Vainio

LIST OF ABBREVIATIONS AND SYMBOLS

OUSPG	Oulu University Secure Programming Group
PC	Personal Computer
GUI	Graphical User Interface
ASCII	American Standard Code for Information Interchange
CPU	Central Processing Unit
UI	User Interface
SUT	System Under Test
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
PID	Process Identifier
DoS	Denial of Service
JSON	JavaScript Object Notation
I/O	Input/Output
2D	Two-dimensional
3D	Three-dimensional
CSV	Comma-Separated Value
TCP	Transmission Control Protocol

1. INTRODUCTION

The use of different kinds of computer security tools has become commonplace in today's world. Most PCs (Personal Computers) are equipped with proprietary anti-virus, anti-malware and firewall tools, and some operating systems even provide such functionality automatically right from the start. The reported software flaws per year have increased to several thousands in the 21st century [1]. When browsing technology news sites, it is not uncommon to see frequent reports of some new data breach, where users' account details were compromised, or where a new vulnerability was found in some frequently used mainstream application. For example, a recent article [2] describes a vulnerability that has persisted in a company's product, a game engine, for ten years while new versions of the engine have been developed and released with the same vulnerability. Recently, the rise of networks and mobile applications has brought new security challenges that were not a concern previously. Mobile app markets, such as Android and iPhone, have created new and popular platforms for distributing software products that may have a great number of new additions and updates on a daily basis, each of which might contain potential vulnerabilities or exploits [3]. When such a large number of potential software testing targets exist, the need for automatic test processes becomes apparent.

Fuzz testing, or fuzzing, is a form of software testing, which aims to find flaws in software by providing to it random or semi-valid input data in order to generate crashes. Fuzzing is usually a largely automated testing method for finding vulnerabilities, but some human interaction is still needed at certain points of the testing process. Analyzing the results while and after fuzzing is important, in order to find out which crashes and behaviors are especially troublesome. Monitoring one or a few test applications is a simple enough process that can be done from the command line or by using the GUI (Graphical User Interface) that some modern fuzzers provide. For example, the web interface of the fuzzer framework Sulley, shown in Figure 1, displays a testing process, which has generated one crash and is currently running test case #43. The web interface offers the same amount of visual information that could also be shown in a text console: ASCII (American Standard Code for Information Interchange) progress meters and colored text.



Figure 1. The web interface of the Sulley fuzzer.

In addition to monitoring a single program that is executing test cases, there could be benefits to be able to change the scope from monitoring the fuzzer application to monitoring the whole testing machine itself. This would enable the gathering of different performance measurements from the test computer and examining them together with the progress of the testing process. Furthermore, there are currently companies and groups that are using up to thousands of CPUs (Central Processing Units) to run fuzz tests simultaneously. Being able to monitor multiple automated fuzz test machines with an intuitive visual UI (User Interface), which gives important information about the testing process at a glance, could prove to be a useful development.

There are some general monitoring solutions, such as Nagios and Icinga, that can simultaneously monitor a large number of test computers and provide information about the systems and their services [4]. Figure 2 shows the web interface of the Icinga monitoring software, and demonstrates how quickly it can be inferred that something out of the ordinary is happening, when the first thing noticed is the orange Warning box amidst the green OK boxes. Humans are adept at seeing outliers and patterns in visual form, therefore utilizing more elaborate visualization techniques could benefit the monitoring process even more. Additionally, in the case of fuzzing, a more specific monitoring tool that could synchronize different performance data of the test computer with the resulting data obtained through fuzzing could be beneficial.

[View History For all hosts](#)
[View Notifications For All Hosts](#)
[View Host Status Detail For All Hosts](#)

Display Filters:

Service Status Details For All Hosts

Host ^v	Service ^v	Status ^v	Last Check ^v	Duration ^v	Attempt ^v	Status Info
localhost	Current Load	OK	08-13-2011 18:50:26	0d 2h 25m 48s	1/4	OK - load average
	Current Users	OK	08-13-2011 18:51:04	0d 2h 25m 10s	1/4	USERS OK - 2 u
	HTTP	WARNING	08-13-2011 18:49:41	0d 2h 24m 33s	4/4	HTTP WARNING
	PING	OK	08-13-2011 18:47:19	0d 2h 23m 55s	1/4	PING OK - Pack
	Root Partition	OK	08-13-2011 18:47:56	0d 2h 23m 18s	1/4	DISK OK - free s
	SSH	OK	08-13-2011 18:48:34	0d 2h 22m 40s	1/4	SSH OK - Open
	Swap Usage	OK	08-13-2011 18:49:11	0d 2h 22m 3s	1/4	SWAP OK - 100
	Total Processes	OK	08-13-2011 18:49:49	0d 2h 21m 25s	1/4	PROCS OK: 21

Figure 2. A part of the Icinga web interface.

This thesis presents a monitoring tool that uses data visualization techniques to present both real fuzz test data and simulated scenarios of multiple fuzzers running simultaneously. The performance metrics and fuzzing data are collected from a target remote host and different visualization methods are used to depict the data side by side. The aim is to inspect the research question: “What kind of benefits does using different visualization techniques bring to the monitoring of fuzz testing processes?”

2. FUZZING

Fuzzing is a testing method which aims to find system errors and exceptions by entering unexpected data into the SUT (System-Under-Test) and by analyzing the results for faults [5 p.22]. The input data is considered to be semi-valid in the sense that the tested application accepts it initially but still malformed enough to potentially cause faults [6]. Fuzzing is considered to be a form of negative testing which tries to find anomalous and unexpected behavior in the test subject, instead of testing for correctness or performance [7 p.17-19].

Fuzzing originated as a heavily randomized black-box testing method, where no knowledge about the system's internal structure is assumed [8]. Although the early approach was considered to be naïve, it provided impressive results at the time [9]. In practice, it is reasonably easy to write a simple and practical fuzzer that finds at least superficial bugs in some applications. Fuzzing is also thought to be relatively efficient and inexpensive testing method [10]. In fact, a part of the appeal of fuzzing is that it does not need much manual control of the testing process; instead it is possible to automatize a significant portion of the testing process to cover a large amount of test cases [6].

2.1. Phases of Fuzzing

The overall fuzz testing process can be divided into several phases [6, 5 p.27-30]. One way to visualize the process is shown in Figure 3.

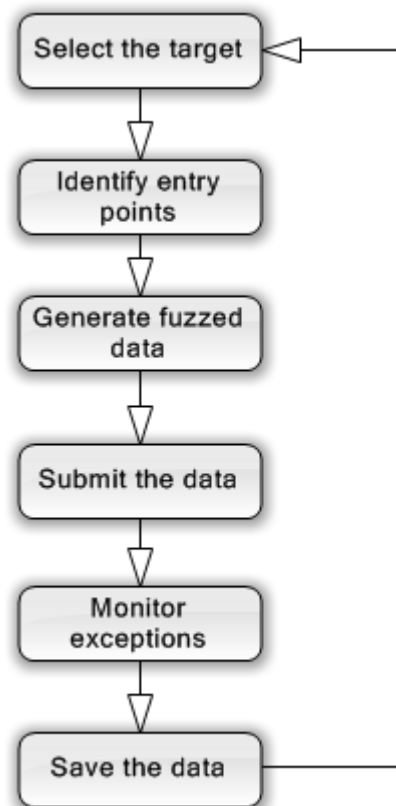


Figure 3. The basic phases of fuzz testing.

The first phase is usually the process of identifying which target to test. Fuzzing can be applied in a number of domains. The targets can vary from the simplest command-line applications to file formats and network protocols [5 p.36-44].

After selecting the SUT, the next step is considering what kind of input data to fuzz [5 p.27-28]. There are various entry points in different kinds of software systems where input data is entered into the system, and their type of input mostly depends on the fuzzing target [6]. The most common entry points are usually files, registry entries, Application Programming Interfaces (API), UI, network interfaces, database entries and command-line arguments [6].

The third phase consists of using automated tools to generate the data according to the selected input vectors [5 p.28]. There are multiple different methods to generate the data, but a special attention needs to be placed on the fact that the data set which is generated should have mostly correct data, with pieces of malformed or semi-valid data [6].

After generating the input vectors they are submitted to the SUT, in an automated fashion. For example, in the case of network fuzzing it could be the process of sending data packets, or in the case of desktop applications it could be opening a file which contains the input test data. [5 p.28]

The last two phases consist of monitoring the testing process and saving relevant data for later use. Often it is not enough to just cause a crash in the tested program, but there should be a way to locate the input vector that actually caused the crash [5 p.28]. All the input vectors that cause a failure should be automatically saved for later inspection, which often occurs manually [7 p.31]. As Figure 3 shows, the saved data can also be used to construct additional test cases.

2.2. Types of Fuzzers

One of the main concerns in generating fuzzer test cases is the input source of the malformed data. Fuzzers are often categorized according to the method by which they obtain the input data, i.e. either by generation or by mutation [7 p.137]. They can also be classified according to the classical black-box and white-box definitions of software testing [11], or by the terms intelligent and non-intelligent [7 p.137].

In generation-based fuzzers the input data vectors are created from scratch according to a model or a specification [6]. The specification could be, for example, a file format definition or a protocol description [5 p.33]. One potential process workflow could be to create a grammar to describe the protocol specification and let the fuzzer to generate the fuzz data according to the definitions [5 p.36]. Generation-based fuzzers are often largely dependent on the language and scenario specifics [6, 7 p.137] and because they require prior knowledge about the system they can be classified as white-box testing methods [3].

In contrast to generating the data entirely by itself, mutation-based fuzzers use a set of valid data as a source and mutate it in certain places to obtain malformed data [6]. As an example, where a generation-based fuzzer would generate malformed HTTP (Hypertext Transfer Protocol) messages according to their specification, a mutation-based fuzzer would instead use existing HTTP messages and transform them to the malformed input vectors. Mutation-based fuzzers are considered to be general purpose fuzzers [7 p.137], because they do not necessarily need to conform to a specific scenario and environment. In their basic form mutation-based fuzzers are called brute force fuzzers, because their main purpose is to modify data and pass

it along without much intelligence [5 p.36]. This kind of approach does not require to have any prior knowledge about the SUT, and is therefore considered to be a black-box testing method.

The choice of whether to use generation-based or mutation-based fuzzer depends largely on the scenario. When good data sources, such as files and interfaces, are easily available, it seems to be logical to choose mutation-based fuzzers. They also have the added benefit of often being easier and simpler to implement. On the other hand, sometimes it can be difficult to obtain good input sources to mutate, and therefore the generation-based approach may be more fitting. Additionally, certain scenarios with strict format definitions, such as network protocols, can be easier to test with generation-based fuzzers. [6]

2.3. Fuzzing Target Monitoring

Generating malformed data and making a target system crash is an important part of fuzz testing, but it is also important to monitor the target and how it reacts to specific inputs. There are several different kinds of methods for monitoring fuzzed systems, and [7 p.170] lists some of the most important ones.

2.3.1. Valid Case Instrumentation

The most basic way to monitor the status of the SUT is to use continual checks that determine whether the system is still functional. In practice, this could mean sending a confirmation message between each test case to specify that the system is still available, such as is seen in Figure 4.

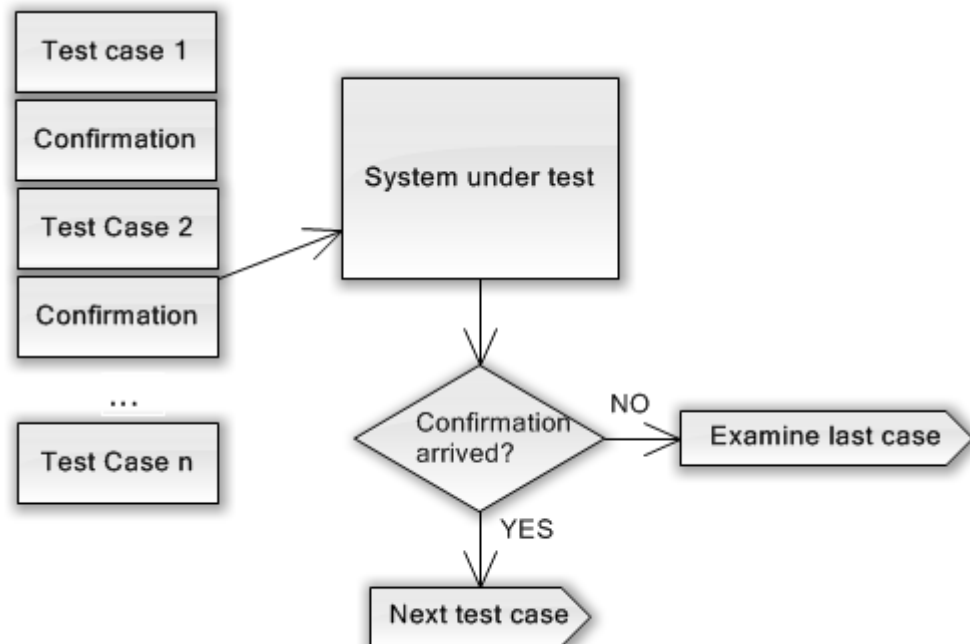


Figure 4. Valid case monitoring.

If the message does not come through, then the test case that was run last should be examined. A more advanced method would be to monitor that the system still produces a correct response to a valid input between test cases. Valid case monitoring is relatively simple to implement but it has some considerable drawbacks. For example, if a crash occurs in a certain child process that was spawned by the main service that is under testing, it may not necessarily cause the service itself to become unresponsive. [7 p.171]

2.3.2. *System Monitoring*

A more powerful option for target monitoring could be to monitor the system under which the application is running, instead of just the application itself. There are many ways to gather relevant monitoring data from a system. One of the first things to check could be the log files provided by the system or the application. Some applications provide detailed logs that can be monitored for faults, often with detailed data about an unexpected behavior, such as the time of the crash or the Process Identifier (PID) of a faulty child process. [7 p.172]

Different performance statistics, such as CPU utilization and memory usage, could also provide important information about the fuzzed system. If a CPU spike occurs during testing, it might indicate that the program is using an intensive algorithm that has some bounding issues [6]. Notable changes in memory consumption or CPU utilization could also indicate a potential Denial of Service (DoS) attack [7 p.175]. If networking is an important part of the SUT, it might also be reasonable to monitor the network usage of the system.

2.3.3. *Monitoring With a Debugger*

One of the more advanced forms of monitoring is to attach a debugger to the program that is under testing [7 p.176]. Debuggers are software tools that can be used to detect faults, exceptions and interrupts that occur during the application execution. Typically debuggers allow the user to control the code execution by setting breakpoints and by executing the program code step-by-step, in order to find the exact location of the faults [12 p.2-3]. When it comes to fuzzing, it can be more useful to automate the debugger to provide log files when an exception is triggered, and afterwards continue the execution of the testing process rather than halt it. The log files can be later examined to find the exact locations and the type of faults that were found. Monitoring fuzzing with a debugger is especially useful for detecting memory corruption bugs, but on the other hand, every occurrence of memory corruption may not result in an exception being thrown [7 p.178].

2.4. State of the Art

While early fuzzing implementations used heavily randomized input data, that approach is currently seen as ineffective for fuzzing real-world programs [3].

In order to produce effective test cases that have good coverage, the input data vectors should pass the SUT's input validation phase [13]. Currently many protocols and file formats have implemented checksums to verify the integrity of the input data [6]. These validation techniques may cause a large number of malformed inputs to be rejected at an early stage of testing [14].

The black-box and white-box fuzzers both have their drawbacks. Strict white-box fuzzing needs the source code to be available, and black-box fuzzers lack input selection that is dependent on the testing process that is occurring [15]. A number of developments have been created to find solutions to these issues.

Firstly, the use of genetic algorithms has been proposed to provide "intelligence" to a fuzzer that has no access to the source code of the SUT. By using techniques such as *grammatical evolution*, the best inputs that are obtained during the testing process are used to select certain characteristics to maximize the number of constraints in the execution path later in the test process. [15]

Another advanced fuzzing technique is called *symbolic execution*. Its general idea is to use symbols as input instead of concrete data. When a symbolic value causes the program execution to branch, specific constraints called *path conditions*, are recorded for each path. After a path terminates or a crash is detected, the path conditions leading to that event can be solved to provide concrete inputs, which satisfy the constraints and lead to the same crash. [16]

The symbolic execution technique can also be adapted to be used in the development of *checksum-aware fuzzers*. Their purpose is to bypass the target programs that use checksums to verify the integrity of their inputs. Normally such checksum mechanisms may render blind input fuzzers ineffective, but fuzzers that can dynamically locate the input data integrity checks and bypass them can improve the effectiveness of fuzz testing. [14]

Gray-box fuzzing is sometimes used as an umbrella term for a variety of fuzzers that gather information about the system and its behavior during run-time, by using analysis and test case adjustment. The term *gray-box fuzzing* comes from its place as a sort of a middle-ground for blind black-box fuzzing and white-box fuzzing that requires complete knowledge of the SUT. [11]

The recent trend of fuzzers to affect the SUT during run-time, suggests that one of the main focuses of current fuzzer development is gray-box fuzzing. One of the reasons why modern fuzzers are more sophisticated in the generation and selection of input data than early fuzzers is their ability to gain better understanding of the target programs dynamically. This is accomplished, for example, by using techniques such as binary analysis and dynamic tracking of code execution graphs during run-time. Additionally, other possible future developments can be expected in the area of monitoring systems, especially to reflect the advances in gray-box fuzzing. Monitoring systems could be developed further to display new types of flaws, such as logical branching errors and security violations. [17]

3. DATA VISUALIZATION

The amount of data has increased at a rapid pace since the transition to a digital age [18]. Not only has the amount of data increased, but also the complexity of it. Data itself, when thought as numbers and text, can be difficult to comprehend as raw values [19]. Data visualization is a discipline which aims to represent the data in such a way that a human can analyze it and gain insight into it more easily.

The basic principles of visualization have existed for a long time. The earliest uses of visualization were drawings of geometric diagrams and maps that were used in the early 15th century. The field of data visualization can be said to have started as measurement instruments and technologies started to develop and improve in the 16th century. As fields such as analytic geometry, statistics and probability progressed in the 1800s so did the visualizations depicting their phenomena. In the modern world, the use of computers to draw the visualizations has created totally new visualization paradigms that were not possible in the past. As the amount of data has grown through the history to current day, so has our means to visualize it [20]. In the past few decades, the data visualization field has gained use in a discipline called exploratory data analysis, which is used to analyze datasets to find their main characteristics [21].

The benefits and reasons to use visualization are many, and there have been thorough analyses such as [22] that have proven the usefulness of visualization. The human visual system comprises nearly half of the brain and visualization utilizes its ability to obtain a large amount of information at a glance. Not only that, but the human brain has an innate ability to detect patterns and identify relationships in visual information. Therefore visualization is especially good at identifying trends, gaps, outliers and points of interest in a data set [23]. One classical example demonstrating the usefulness of visualization in analyzing data is called the Anscombe's Quartet [24] as seen in Figure 5. The datasets look similar when analyzing their statistical properties, such as mean values, variances and correlations, but have considerable differences when visualized.

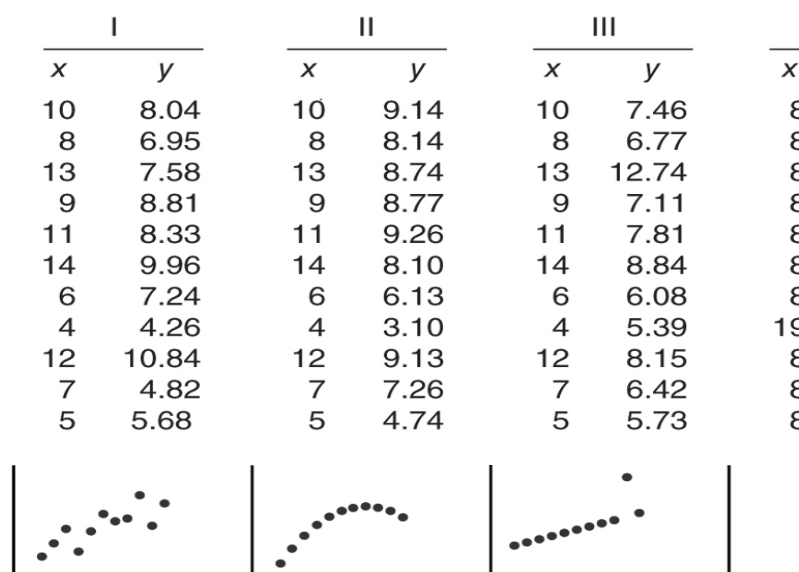


Figure 5. A demonstration of the Anscombe's Quartet.

Visualization methods can be categorized in a few different ways. The broadest category is called *information visualization*, which has to do with visualizing almost anything, such as tables, graphs or even text. Currently though, the term information visualization has changed to mean visual representations of large amounts of non-numerical data, such as lines of code. Another more specific category is called *scientific visualization*. Its primary focus is in visualizing three dimensional phenomena such as medical, biological or meteorological models. Finally, there is also the slightly narrower domain of *data visualization*, which deals with representing different kinds of data, which usually in this context is defined as some form of abstracted units of information, such as attributes or variables. [20]

The process of creating visualizations from an underlying dataset contains several steps. Firstly, the data in the visualization scenario must be collected or generated to form a set. After obtaining the dataset, it needs to be parsed into a format that provides some meaning or categorization which is relevant to the scenario. Next, the relevant portions of the data are identified and the rest are discarded. Finally, a visual model, which is considered to fit the scenario and the dataset, is chosen and adjusted to make it comprehensible and aesthetically pleasing. Figure 6 shows some of the most used, basic visual models in data visualization. [25]

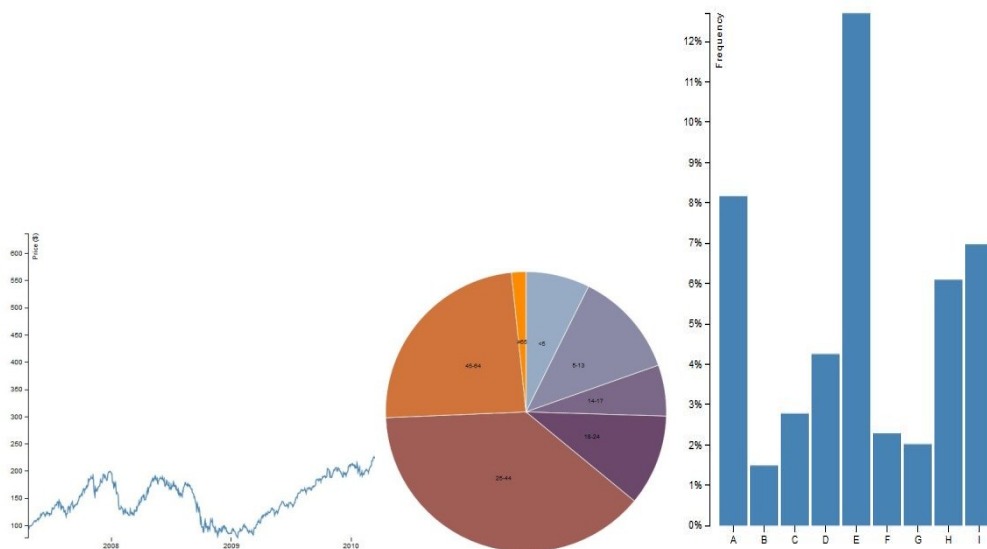


Figure 6. The basic charts: line-, pie- and bar chart.

3.1. Uses of Visualization

The main uses of visualization can be categorized into two main approaches: Explanatory visualization and exploratory visualization [26 p.4]. The differences between the two approaches lie in their main purposes and possibly in their preferred tools and techniques.

3.1.1. *Explanatory Visualization*

The purpose of explanatory visualization is to present an existing dataset to somebody, which is why the term presentation visualization is also sometimes used. The main point is that the designer of the visualization already knows the dataset and what story the visualization should tell with it. In order to design a good presentation of data, it is important to know the audience and consider what data to display and to highlight to the audience. [26 p.5, 23 p.8]

Explanatory visualizations can be further categorized into three categories: informative visualization, persuasive visualization and visual art. The main goal of informative visualization is to present the data to the viewer in such a way that it informs the reader or helps to get some point across. The intention is to neutrally depict the data, although these types of visualizations are sometimes accompanied by text captions that aim to use the neutral visualization to influence the reader in some way. In contrast, persuasive visualization is its own category, which is specifically used to influence the reader's opinions and views. They are carefully created to support the designer's agenda and to convince the reader to change his mind. Finally, in contrast to conveying useful information, data can also be visualized as an art form to create beautiful graphics. Here, the main concern is not to depict some dataset for the reader to extract important information. In fact, the reader may not understand the underlying data at all from the art visualization. [23 p.8-11]

3.1.2. *Exploratory Visualization*

In contrast to explanatory visualization, exploratory visualizations are not mainly used for presenting findings or conclusions to others. In fact, exploratory visualizations may be seen by only a single person, who is analyzing the dataset [26 p.5]. Drawing visualizations of a partially unknown dataset can be a valuable tool to quickly identify useful information about it, and that is exactly the main purpose of exploratory visualizations. It can be said that a designer uses explanatory graphics to tell a story to its readers, and exploratory visualizations to find a story that the data is trying to tell [23 p.7].

3.2. **Recent Developments**

The key point in the current state of data visualization is that it has largely formed into a multi-disciplinary field. Because the recent developments have occurred across many different related fields, it is difficult to give a comprehensive overview about the current state of the art. Some of the major themes that have occurred during the past 20 years are viewer interactivity, methods for utilizing high-dimensional data and dynamic exploration of the data. [26 p.40]

The addition of interactivity to visualizations has allowed for the development of more sophisticated exploratory visualization methods than previously. For example, visualizations that can be moved and zoomed by brushing, or selected by clicking, can be of great benefit in exploratory visual analysis. [26 p.40-42]

The 2000's saw the rise of new visualization techniques such as *tag clouds*, *sparklines* and *chord diagrams*. [27]

The tag cloud is a visualization technique that depicts the frequency of the most commonly used words within a text selection. It does so by drawing the most used words larger than the others, as seen in Figure 7. Tag clouds have been used, for example, as navigation aids in websites to display different tag categories for articles.



Figure 7. A tag cloud visualization of a text article, which describes tag clouds.

Sparklines are a type of small line charts, which mainly depict the shape of a variation occurring over time. Sparklines usually appear in line with text and are designed to be used in context of words, images and tables. Parallel sparklines can also be used to quickly provide information about a situation with multiple connected variables. For example, Figure 8 shows how deviations from the normal range of temperature, represented by the gray band, can be evaluated together with the other relevant medical variables to quickly assess the current state of the patient. [28]

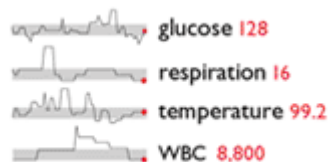


Figure 8. Medical data represented by sparklines visualization.

Chord diagrams are visualizations that utilize a circular layout to better show the relationships between data links. Chord diagrams are used especially in the field of visualizing genomic sequences and their properties, but they can also be used in any other scenarios that involve displaying relationships among groups. For example, Figure 9 shows the dating preferences of people with different hair colors: blonde, brown, black and red. [29, 30]

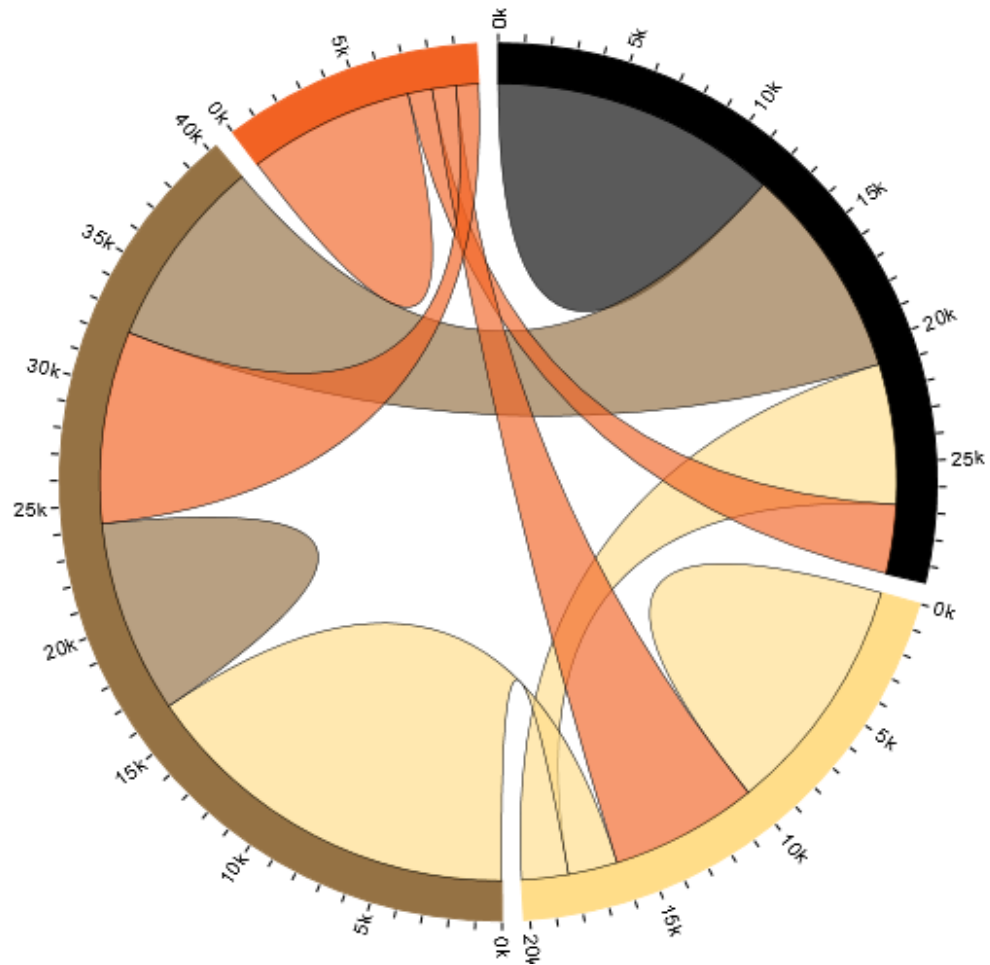


Figure 9. A chord diagram representing preferred hair colors in dating partners of an example population.

3.3. Visualization Methods for System Monitoring

There is a large amount of visualization techniques to choose from, many of which are detailed in [31]. Selecting the appropriate visualization techniques and encoding processes for a design scenario depends largely on the data that is to be visualized [23 p.16]. For example, three specific types of visualization methods could be considered to be useful when creating a system monitoring scenario: hierarchy representation, multivariate dataset visualization and parallel comparison of multiple time series.

Firstly, if the monitoring system displays data from multiple nodes, each with their own specific groups and multiple parameters, there should be a way to represent that hierarchy for an overall view of the monitoring scenario. Enclosure diagrams, such as *treemaps* and *circle-packing layouts*, are examples of visualization techniques that are especially useful when representing a hierarchy of a system [32]. An example of a treemap visualization is shown in Figure 10, in which colors are used to distinguish different node groups, and the area size of individual nodes is used to represent their values.

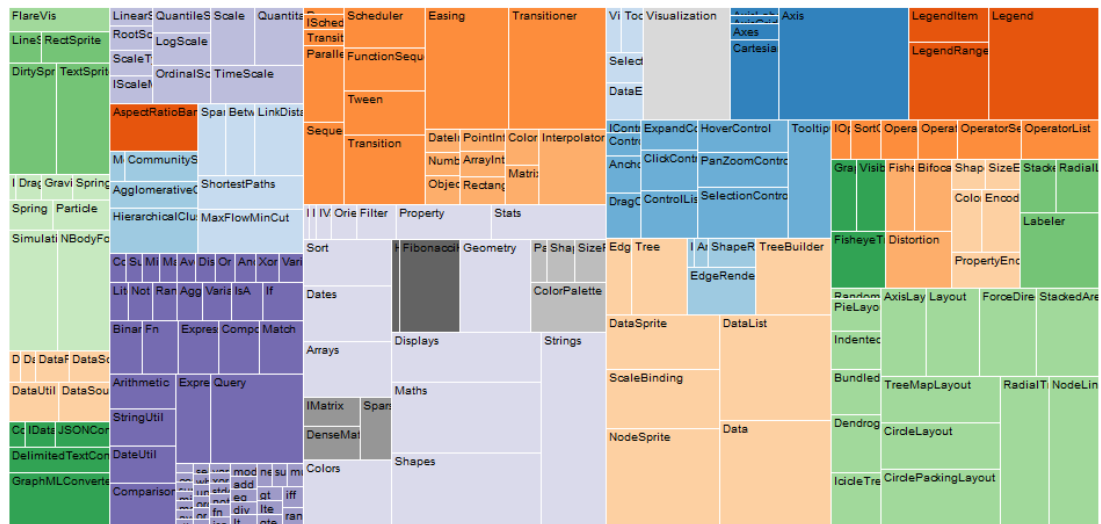


Figure 10. A treemap visualization representing the hierarchy of a system.

Treemaps use space efficiently to show containment of data nodes and their sizes, while circle-packing layouts are less space-efficient but are easier to understand visually [31, 33, 23 p.56]. Figure 11 shows a circle-packing visualization example, which uses colors to depict different file types and the radii of the circles to represent the individual file sizes.

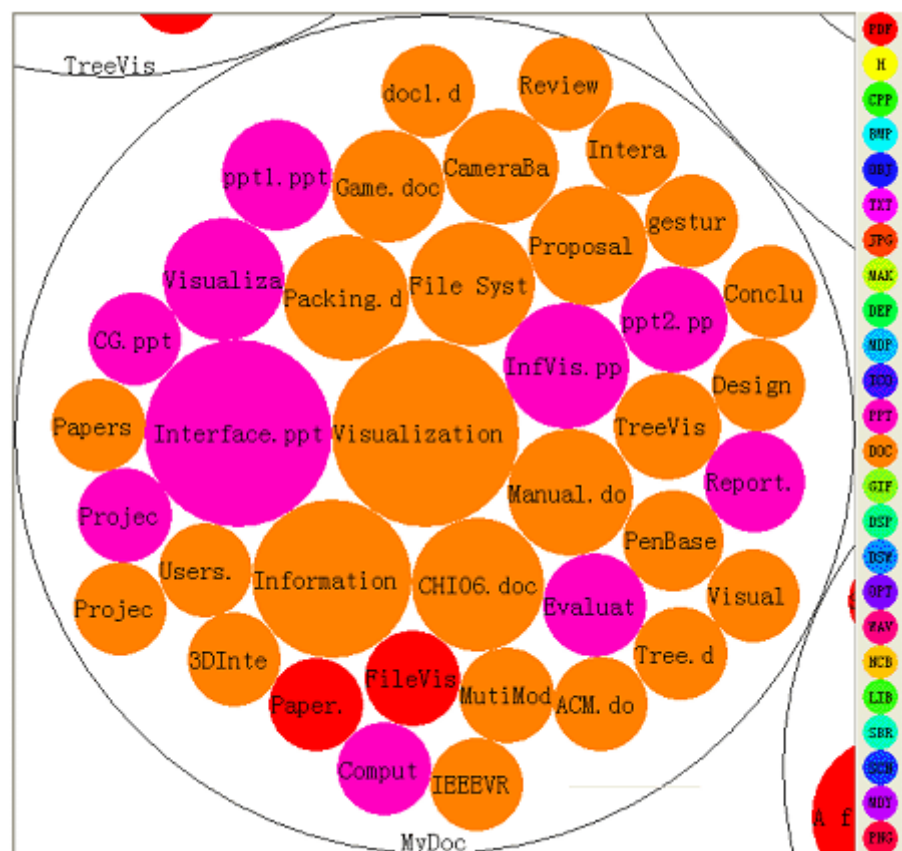


Figure 11. A zoomed-in circle-packing visualization depicting files in a directory.

The second important point to consider in system monitoring visualization is how to represent multivariate data and its relationships. Although multivariate data is considered to be difficult to represent, there are two effective visualization methods for doing so: the *scatter plot matrix* and *parallel coordinates* [32]. Figure 12 displays both of the techniques visualizing the same data set, with scatter plot matrix on the left and parallel coordinates on the right. The scatter plot matrix uses multiple scatter plots for showing correlations between pairs of variables, whereas the parallel coordinates technique draws the data on parallel axes with lines connecting the data points [32].

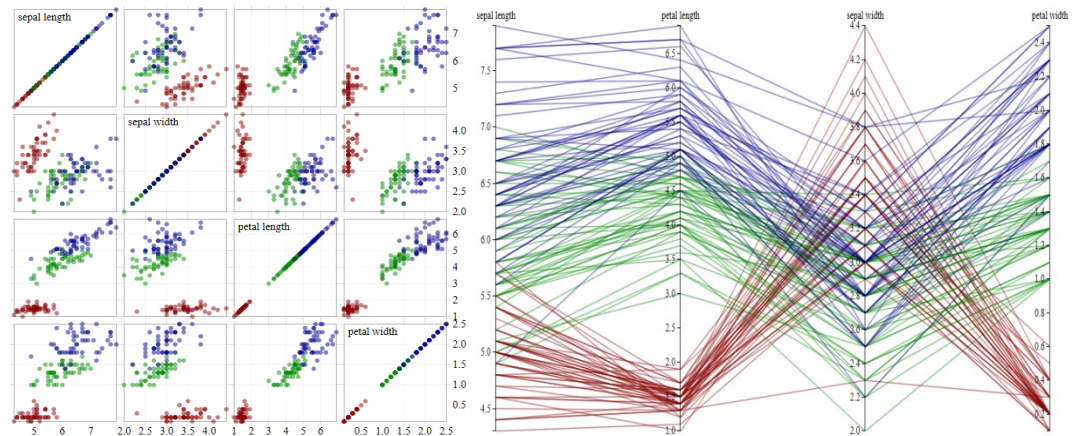


Figure 12. Scatter plot matrix and parallel coordinates.

Finally, an important part of a system monitoring scenario can be thought to consist of value comparison tasks of basic time series data, such as for example memory usage over time. It is possible to use basic visualization techniques, such as line charts, to do so but in the case of visualizing a large number of time series simultaneously, effective use of space can become a problem [34]. One method designed to address this problem is a visualization technique called *horizon graph*, which aims to increase the data density of time series line charts, yet still allowing viewers to easily compare data across multiple graphs [32, 34]. The horizon graphs are constructed by dividing a basic line chart into layered bands and by offsetting and mirroring negative values with positive values, as seen in the example in Figure 13.

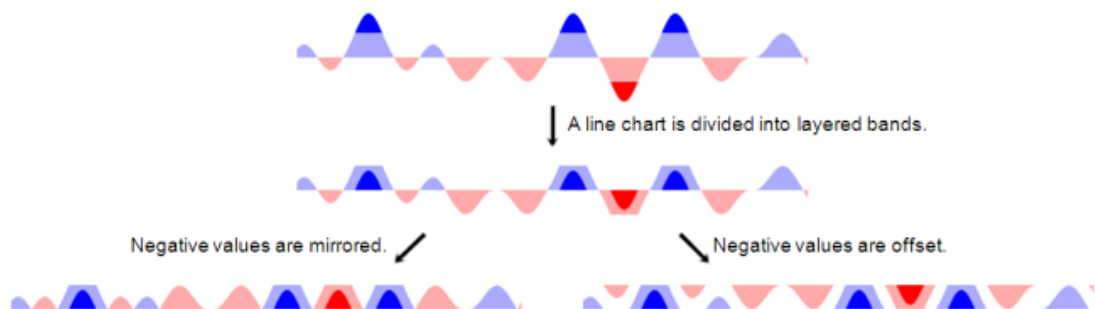


Figure 13. The construction of horizon graphs.

4. DESIGN AND IMPLEMENTATION

The design and implementation part of this work consists of developing a monitoring server that uses different visualization techniques to represent datasets in two distinct scenarios, in order to analyze the research question outlined in Chapter 1. In the first scenario, a single target system is equipped with a fuzzer and a performance measurement collector to obtain the test data. Next, the data is sent over the network to the monitoring server and afterwards it is visualized in a web browser. The second scenario examines the use of data visualization in the context of a large number of multiple fuzzers running simultaneously. In order to focus more on the visualization aspect of this work, the fuzzers are implemented as simulations in pre-defined input files, which are created beforehand or during run-time. While it could be possible to install a large number of systems, with each running multiple fuzzers, for the scope of this thesis the simulation approach was chosen.

One of the main points to consider in this work is the question of what kind of data should the monitoring system obtain and visualize. As some fuzzing experts discuss, such as in [6] and [7], gathering CPU and memory statistics can be very useful to understand details about the testing process. Some consideration could also be given to monitor power consumption in order to possibly visualize valuable information about the efficiency of certain fuzzing operations. Being able to visualize crashes found over time and the overall power consumption could potentially help compare which fuzzers and test cases are better from an economic standpoint.

The development process for the monitoring system scenario can be divided into two main parts, each with their own requirements. Firstly, there should be a central monitoring server that does the data parsing and visualization. It should provide a graphical web interface that can be used to examine the visualized data. Secondly, there should be the possibility of connecting multiple hosts to the central server. Each of the hosts should be running data collector daemons and on-going fuzz tests, while sending their results to the central server in real-time. In conclusion, the overall structure of the system is a client-server architecture as shown in Figure 14.

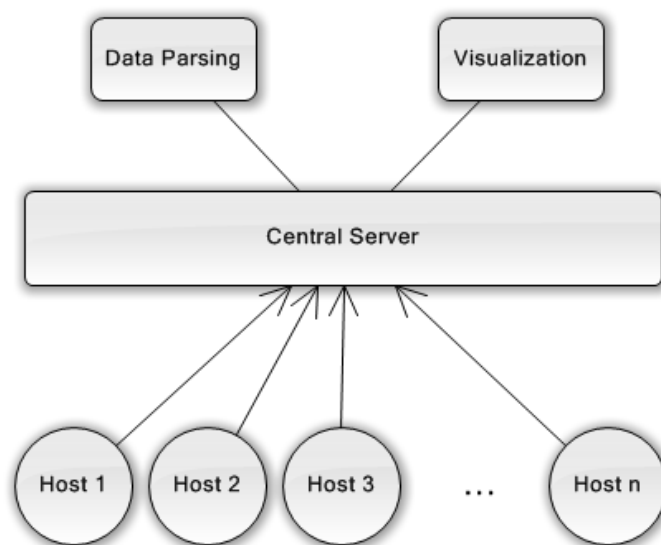


Figure 14. The overall structure of the system.

The main requirements of the implementation part of this thesis can therefore be divided into four parts: data collection, fuzzer implementation, networking and data visualization. The monitoring server handles the networking and visualization parts, and the remote hosts are used for the data collection and fuzzer implementation.

4.1. The Monitoring Server

The monitoring server has three main functions to perform. Firstly, the server is capable of receiving data into its listening socket, and if the data received is a valid JSON (JavaScript Object Notation) file with a correct form, the server application saves the data into a database text file. Secondly, the server also hosts a web server separate from the listening socket, in order to provide a web page interface for the software. The data visualizations are created on the web page by using a specified visualization framework.

4.1.1. Web Frameworks

There were many different web programming frameworks to choose from for this scenario. Apache [35] and Django [36] are two of the well-known frameworks that were considered but ultimately Node.js [37] was chosen, because of its perceived scalability and minimized overhead that result from its event-driven asynchronous I/O (Input/Output). It also provided good ready-made libraries for this task.

The Node.js framework has two main tasks to perform: to obtain the data from the remote hosts and to host a webpage to visualize the data. Node.js has its own package manager, *npm*, which contains many kinds of modules for different kinds of web scenarios. The two modules that were chosen for this scenario were Socket.IO [38] and Express.js [39].

The WebSocket standard was chosen as a protocol for sending the data files to the monitoring server. The main reason for choosing the protocol was that according to [40], WebSockets should provide a simplified way to manage full-duplex communications over the web and therefore reduce complexity. The WebSocket standard is implemented in Node.js by the Socket.IO library.

In addition to the socket connections, the second functionality of the Node.js server application is to host the webpages that contain the user interface and the visualizations. Instead of using the low-level functions of the Node.js API to serve the web content, the Express framework was chosen to be implemented for this scenario. One of the reasons for this choice was the compatibility between Socket.IO and Express that made it easy to implement them together in one application.

4.1.2. Visualization Frameworks

The task of the visualization framework is to read the received data from input files and to display it in the web browser. The first factor that was considered when comparing different visualization frameworks was the question of whether to use 2D (two-dimensional) or 3D (three-dimensional) visualizations to depict the data. According to [41], 3D visualizations have the benefit of using functions such as

zoom, pan and rotation to help investigate the visualization scenario, but they also introduce a learning curve to the viewer to navigate the data with the camera. Because the main idea in this work was to develop a monitoring server which provides easily and quickly readable charts about the testing process, two-dimensional visualizations were chosen as a more fitting approach.

There are currently many different visualization libraries for the web [42], but for the purposes of this work D3.js[43] was the first choice for several reasons. First of all, D3.js is used by writing JavaScript code, which is the same language as in the Node.js server application. Secondly, D3.js supports several different data input formats, such as CSV (Comma-Separated Value) files and JSON files, both of which are used to store data in the scenarios of this work. Therefore, the need for additional data parsers was lessened. Finally, some benchmarks such as [44], indicate that D3.js is fast compared to other libraries of the same type. D3.js also has good examples and documentation, which showcase its flexibility in creating complex and largely different visualizations. Other visualization frameworks that were considered for this work are detailed in [45], which also argues that when compared to D3.js, some of the other frameworks lack convenient abstractions and can be tedious to use for complex visualizations.

The results of the design choices can be seen in Figure 15, which shows how the main server application developed in Node.js uses the chosen libraries to both host and utilize the visualization framework D3.js.

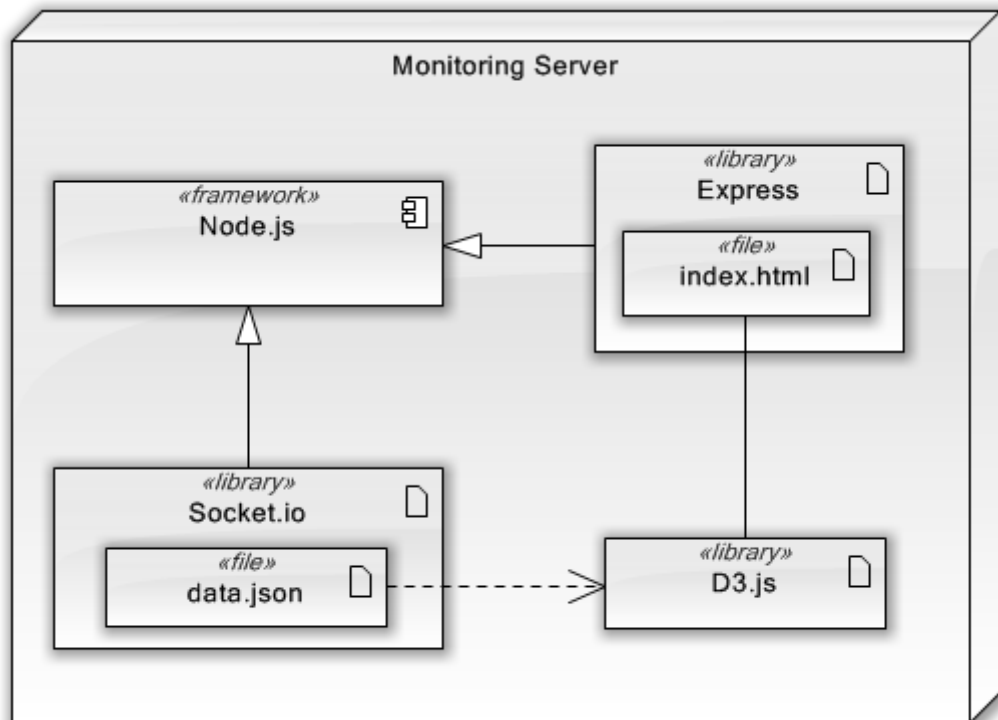


Figure 15. The main components of the monitoring server.

4.2. The Remote Host

The main components of the remote host are a daemon that constantly gathers relevant system data and a fuzzer that provides the testing process data. These two data sources are combined into a single file with a pre-defined format and sent over the network to the monitoring server. The network connection to the central server is established and managed in the same module as the data packing.

When choosing the daemon that collects the performance data, two UNIX applications were examined: Munin [46] and Collectd [47]. Both of them are modular and can be expanded on by plugins and both would have fit the scenario extremely well. Ultimately, Collectd was chosen because it took less time to get it running according to the requirements.

After choosing the system data collector, the next thing to consider was the choice of what kind of fuzzer to implement on the remote host. The fuzzer was required to find at least a reasonable amount of crashes in relatively short time, and to potentially provide some extra data about the testing process that could be interesting to visualize. Several popular open source fuzzers were examined, but finally the easiest and quickest fuzzer to get working according to the requirements was a Bash script that used Radamsa [48] to fuzz the input data from files, which were downloaded from the Internet.

After considering the above mentioned design decisions, the overall structure of the remote host can be seen in Figure 16.

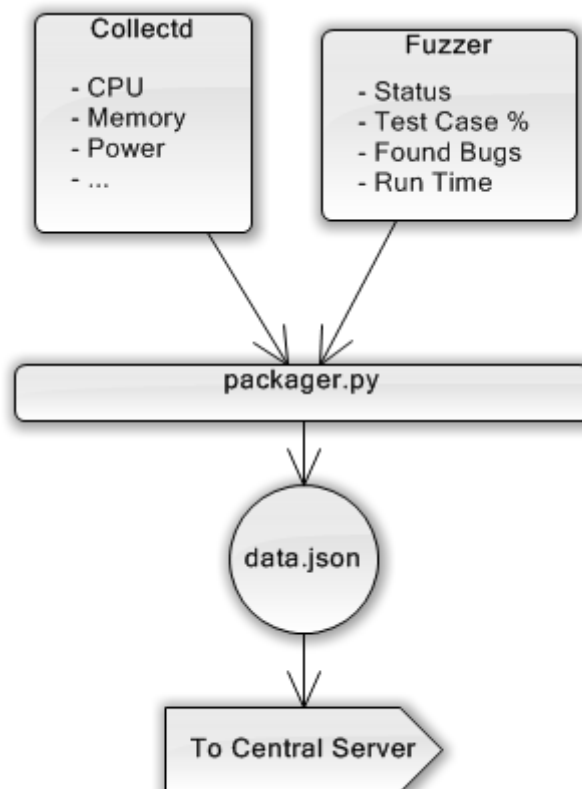


Figure 16. Remote host system architecture.

The Collectd daemon and the fuzzer gather the data and write it into separate files. The packager.py script reads the data files and packages the data into a single JSON format file, data.json, which is then sent over the network to the central server through the WebSockets protocol.

The JSON file was organized in such a way that the D3.js method for loading JSON files can correctly use it as a data source. An example part of the JSON file can be seen in Figure 17. The system name identifier and information about whether the test case was currently running were defined at the top of the file. The rest of the data.json was followed by multiple arrays, with each representing different data to visualize. The arrays included a time in UNIX epoch timestamps and a value associated with that time. For example, Figure 17 shows how each second has a CPU usage value associated with it, ranging from 0 to 100.

```

1 {
2   "systemName": "host1",
3   "testRunning": true,
4   "cpuSystem": [
5     {
6       "epoch": 1378949929,
7       "value": 28
8     },
9     {
10      "epoch": 1378949930,
11      "value": 18
12    },
13    {
14      "epoch": 1378949931,
15      "value": 20
16    },
17    {
18      "epoch": 1378949932,
19      "value": 20
20    },
21    {
22      "epoch": 1378949933,
23      "value": 6.5
24    },
25    {
26      "epoch": 1378949934,
27      "value": 6.5
28    },
29    {
30      "epoch": 1378949935,
31      "value": 52
32    },
33    {
34      "epoch": 1378949936,
35      "value": 6

```

Figure 17. The beginning part of the data.json file.

4.3. Overall System Architecture

After choosing the appropriate tools for the task, the overall structure can be seen in the deployment diagram in Figure 18. The monitoring server is a single device consisting of a Node.js application and a database module for storing the incoming data. The data is recorded into a text file, *found_bugs.txt*, with the following CSV format: *(dtime,hostname,cpu,memory,power...)*. The test computer runs the two daemons, Collectd and packager.py, and the fuzzer executable. The packager.py script implements the WebSockets protocol in Python and sends the packaged JSON messages to the monitoring server every second. The Node.js server receives the data.json file and saves it to the disk temporarily. The JSON file is then parsed for bugs and performance data, and the results are then saved to the database file.

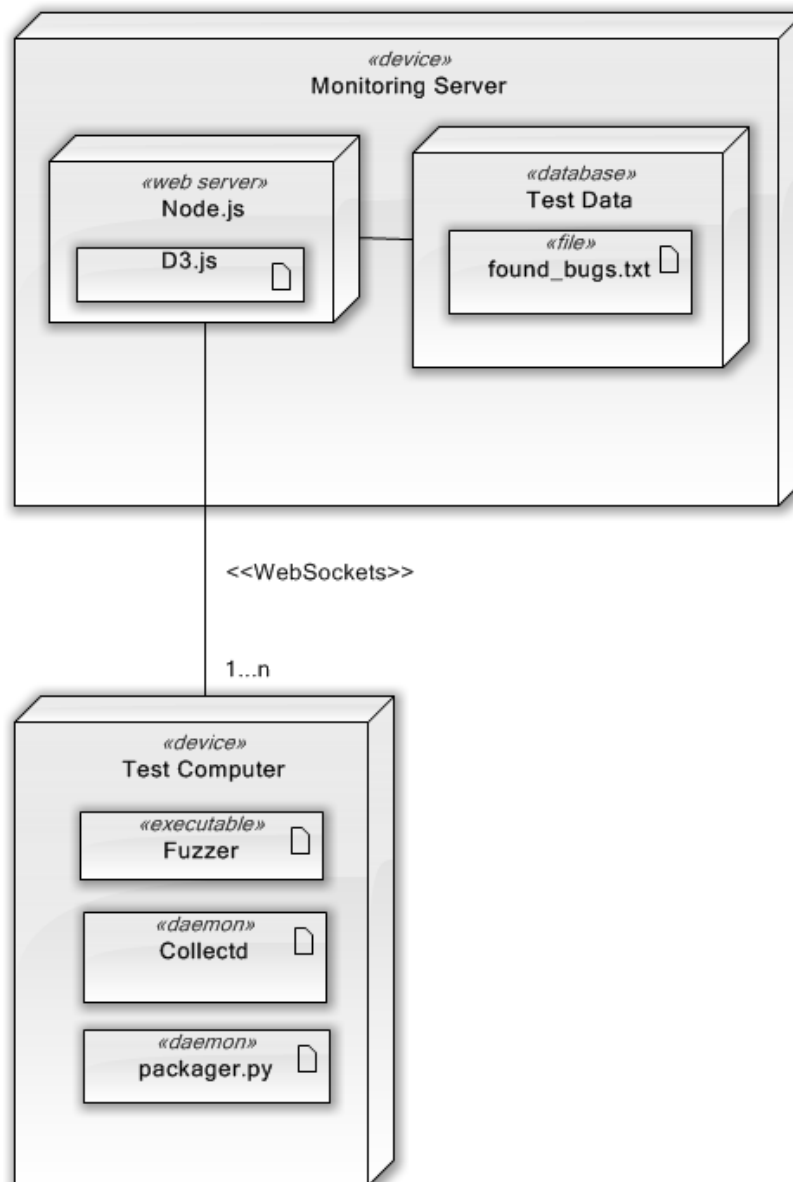


Figure 18. The deployment diagram of the system.

5. CASE STUDIES

The case studies that were performed with the developed monitoring system examined the use of data visualization in both a single remote host and in simulations of multiple hosts running fuzzers simultaneously.

5.1. Case 1: Single Host in an Idle State

In this case study, a single system was connected to the monitoring system and all the relevant performance data was collected while the machine was in an idle state without any tests running. This was done to establish a reference point for later cases.

In the first measurement, the CPU usage of the test host was visualized in real-time to determine its behavior in an idle state. Figure 19 shows a screenshot of the real-time monitoring process that was performed for ten minutes. In Figure 19, the new data input is entering to the graph from the right, and the x axis, which shows time, is moved to the left dynamically. The CPU usage remained stable under 5% during the monitoring.

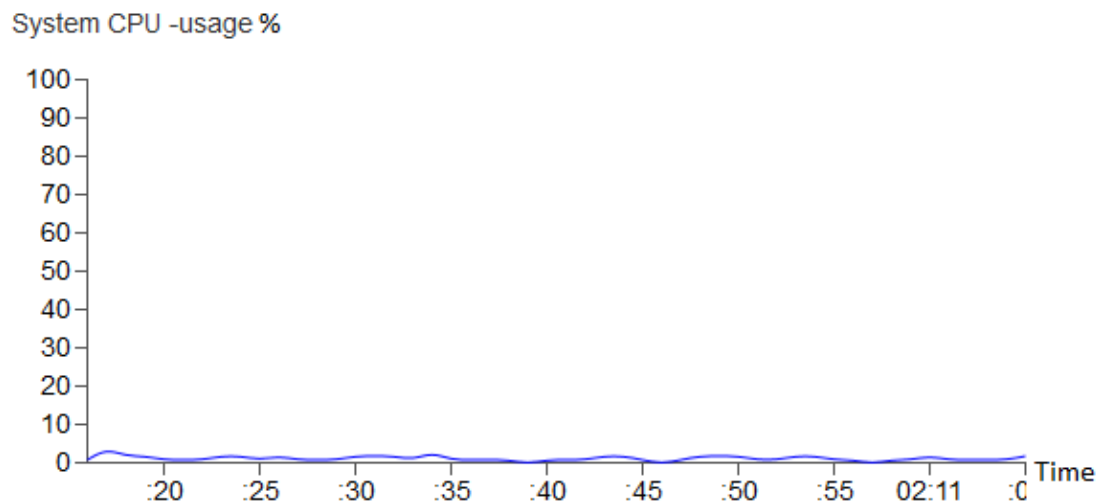


Figure 19. A screenshot of idle state monitoring during the time interval 02:10:15 – 02:11:05.

In addition to the real-time monitoring, the CPU usage was visualized in a larger graph that shows the history of CPU usage over two hours. The CPU usage stayed mostly at under 5%, with occasional 15 - 40% spikes occurring at unpredictable times. Figure 20 displays a zoomed-in part of the graph that shows the highest CPU spike observed during two hours.

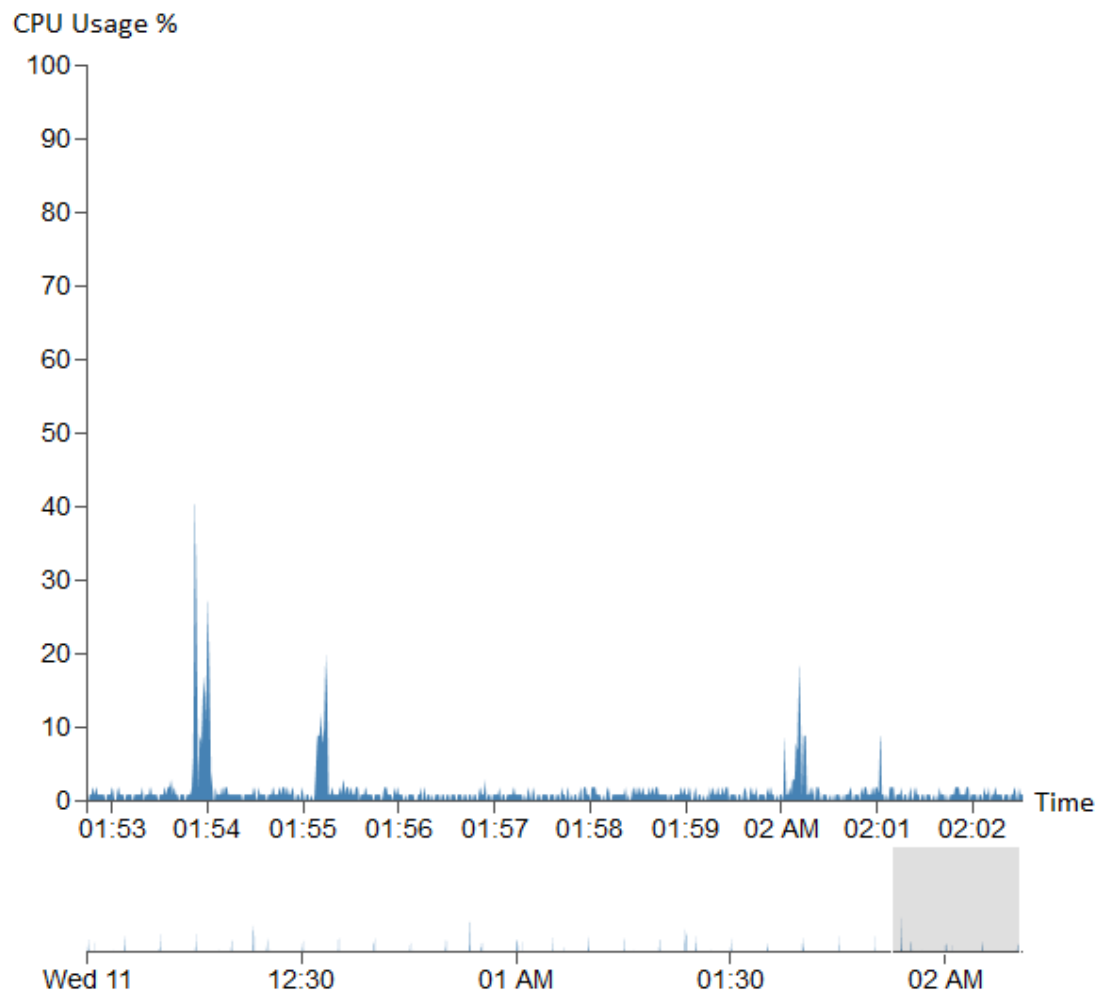


Figure 20. Partial screenshot of the CPU usage history chart.

The memory usage measurement was similarly monitored while the test computer was in an idle state. A zoomed-in part of the memory usage history chart is shown in Figure 21, which shows that the memory usage remained stable at about 23%. Since there were no new processes loaded into memory during the monitoring test, the results were as expected.

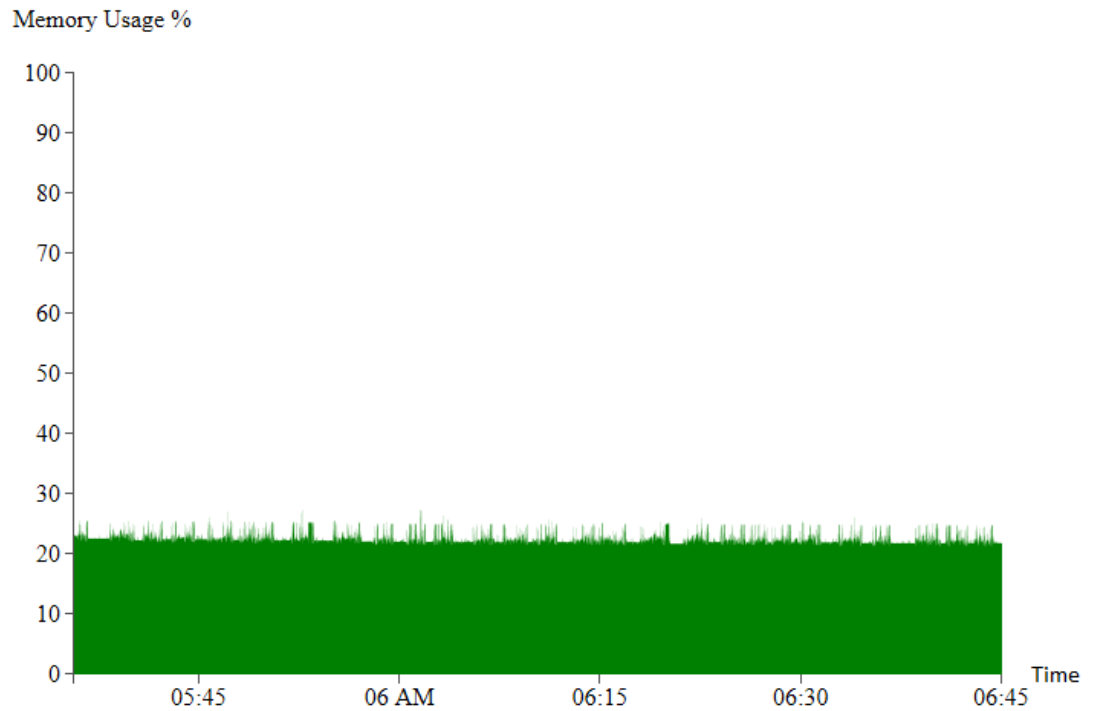


Figure 21. A screenshot of the memory usage history chart showing a zoomed-in time interval.

5.2. Case 2: Single Host under Test

In this case study, a single test host was equipped with a fuzzer that was performing testing on a program, by feeding it fuzzed input files. The fuzzer was kept running for one hour while it recorded all the crashes it encountered during that time. The test computer was monitored through the relevant visualizations both in real-time and after the testing process.

It was expected that the CPU usage graph would show a definite increase at times when the fuzzer is finding crashes or bugs. The testing process was monitored in real-time while following both the CPU usage and the error-found graphs. Figure 22 shows two crashes happening during time interval 02:56:25 – 02:56:30, and the increase in CPU usage before their occurrence.

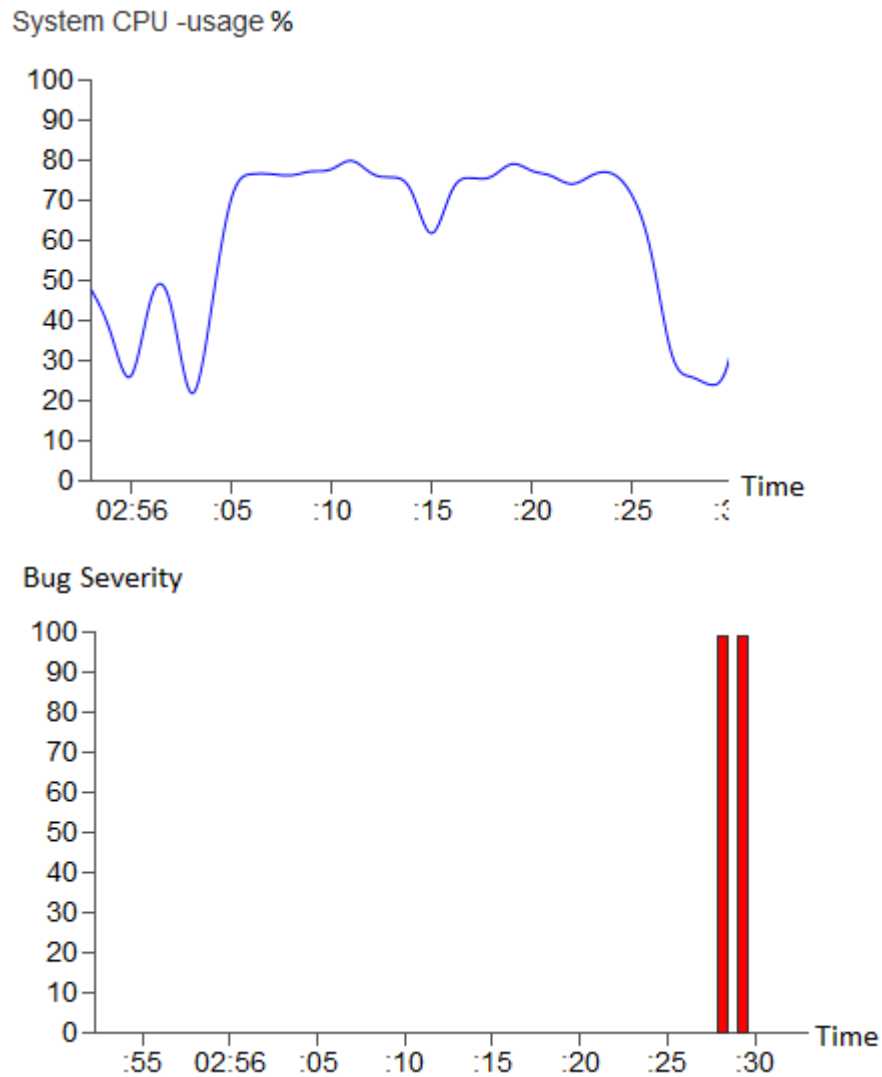


Figure 22. CPU usage graph and crashes-found graph showing two crashes found in quick succession.

When the fuzzer graph showed that something was found at a certain time, the CPU graph was then examined at a time interval that was close to the specified time. The results of 10 such scenarios were recorded into Table 1, which shows the events that happened when the CPU usage was near its maximum value. It can be seen from Table 1, that high CPU usage does not always reliably mean that a crash was found by the fuzzer.

Table 1. High CPU usages and their possible crashes

CPU usage %	Findings	Filetype
80	NO	webm
85	YES	webm
82	NO	webm
85	YES	mp4
80	NO	webm
80	NO	webm
84	NO	webm
40	YES	mp3

The memory usage of the system was monitored similarly and both the real-time monitoring and the complete session charts showed that the memory usage stayed stable even when crashes were found by the fuzzer. This result might be fuzzer dependent, so future research would have to be conducted in that area.

In conclusion, there were definitely CPU spikes happening near the times that the bugs were found. One observation that had to be taken into account was that as the Radamsa fuzzer was creating different input file types, such as mp3 and mp4, different kinds of CPU usage behavior could be expected on different file types. Table 2 displays the input file types and the CPU rates that were observed when generating those files. The results were occurring predictably and had to be taken into account when monitoring found bugs. For example, when generating webm files, the CPU usage would always increase near to the maximum value.

Table 2. Different file type's effects on CPU usage during input generation

Filetype	CPU usage
.ogg	3 – 20%
.ogv	12 – 33%
.webm	50 – 80%
.mp3	5- 20%
.mp4	25 – 48%

5.3. Case 3: Multiple Host Visualizations

In this case study a simulation of multiple hosts running different fuzzers was conducted. The input data files were in a JSON format, which contained all the data that was used to draw the visualizations. The main subject studied here was how to effectively visualize multiple hosts running testing processes, so that the information was conveyed in an easy to understand and useful manner. For visualizing multiple hosts, from 20 to 1000 instances, different visualization approaches were tried and results were examined.

5.3.1. Circle-packing Layout

The first approach was to use a visualization method to represent the overall hierarchy of the monitoring system. In the scenario of this case study, the system environment consisted of five distinct fuzzer groups, each with variable number of remote hosts and test parameters. The circle-packing technique was chosen to represent the hierarchy for this scenario because it seemed to be easier to understand for the viewer than other enclosure diagrams, such as treemaps. The main view of the circle-packing visualization is shown in Figure 23. The largest circle depicts the entire environment of the scenario, and the smaller circles show the branching of the different fuzzer groups and the hosts included in them.



Figure 23. Zoomed-out view of the environment visualized by circle packing.

In order to pack more information into the visualization, a zoom function was developed for the purpose of being able to examine the different single host metrics, such as CPU, memory and newly found crashes. These attributes are shown in Figure 24; where different hosts are zoomed-in on to examine their values. The size of the

circles reflects the values of the appropriate attributes, which makes it easy to make comparisons of different hosts at a glance. For example in Figure 24, Host 3 of the Fuzzer group 1 has greater CPU and Running time values than Host 1, while Host 2 is not running a test process currently at all. Additionally, new bug findings are shown in red text displayed in the Findings circles.

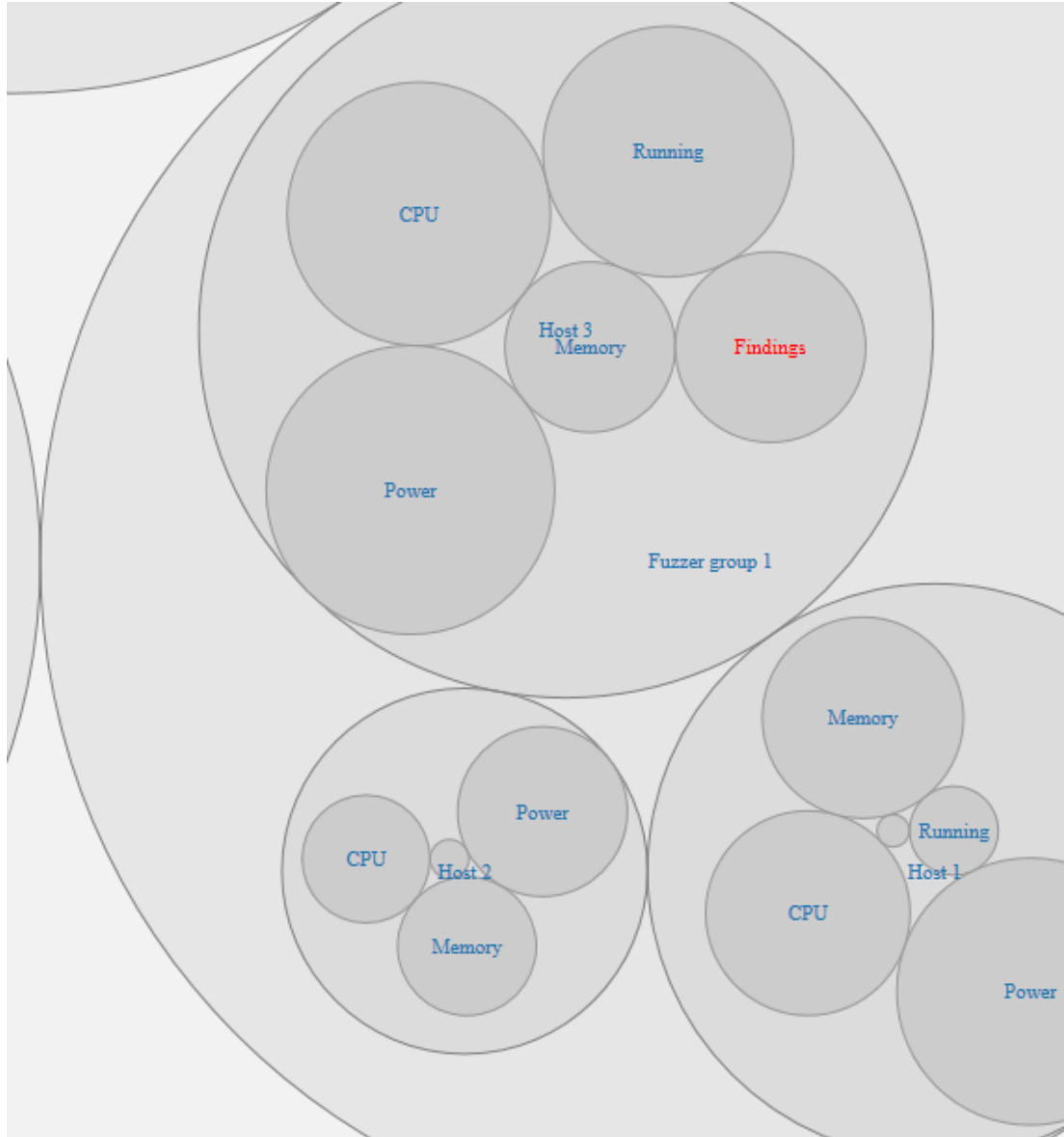


Figure 24. Zoomed-in view on one of the fuzzer groups of the environment.

5.3.2. Multiple Horizon Graphs

In this scenario, single performance attributes of test hosts were chosen to be visualized in a large amount of test hosts in real-time. The main problem was the construction of multiple time series visualizations and how to make useful comparisons between them effectively. The horizon graphs technique seemed to be a natural fit for this scenario, because it enables simultaneous display of a large

amount of time series data in a space-effective manner, without losing much resolution or accuracy.

The input data for this scenario was simulated to show the CPU usage of a 100 different hosts running simultaneously in real-time. Cubism.js, which is a D3.js plugin, was chosen as the main tool for this scenario, because it is an easily adaptable implementation of visualizing horizon graphs that scales well for displaying multiple visualizations simultaneously [49]. While it was not possible to fit all 100 host parameters on the screen simultaneously, still over 20 charts could be shown relatively accurately without scrolling the web page. One of the main features of horizon graphs is to pack the negative values on the same positive axis upside down to save vertical space, but that feature did not help in this scenario because none of the attributes that were visualized had any negative values to display. However, the usual placement for negative values in horizon graphs can be adapted to be used for other data besides CPU usage. Figure 25 displays a screenshot of the simulation process, in which a crash was found by the fuzz host 6 at 6:44 of the running time.

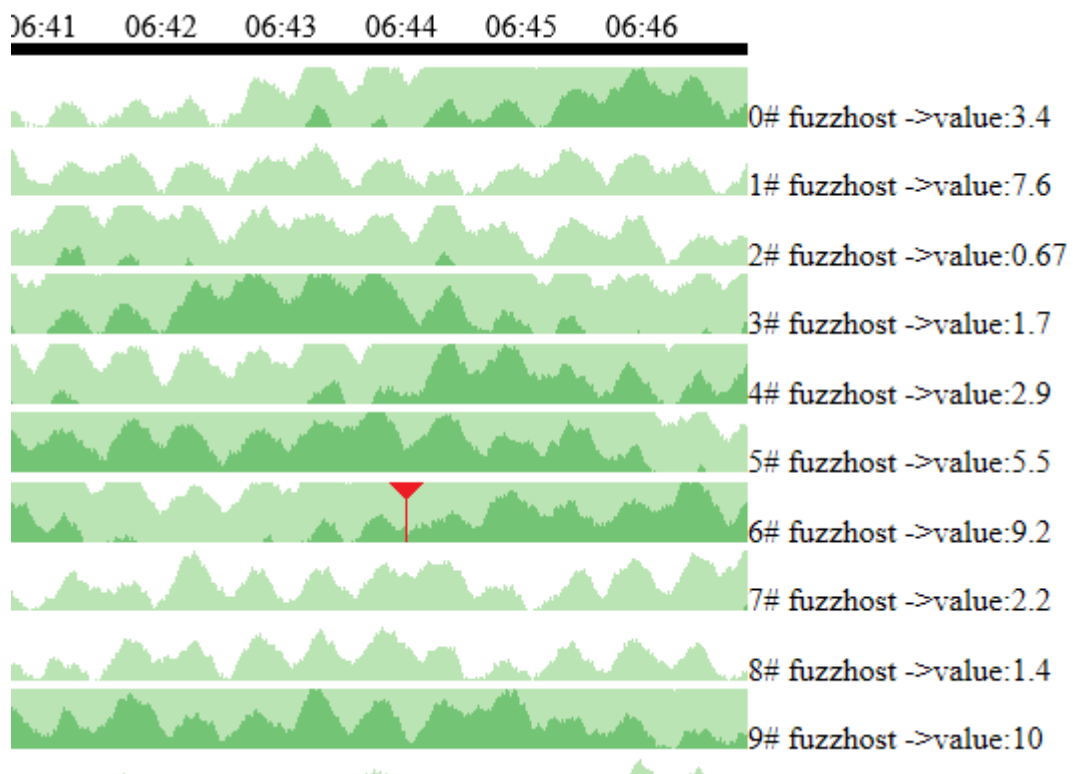


Figure 25. Horizon graphs visualizing a single value across multiple hosts.

5.3.3. Large Multivariate Datasets

The aim of this scenario was to simulate a large number of hosts, such as 1000, and to gain information about them at specific times. The input file for the simulation was a CSV file, which contained all the data of found crashes in a 24h period of fuzzing performed by 1000 hosts. To use visualization in exploring large data sets, it makes sense to provide multiple views to the material to aid in the exploration of the data. Crossfilter [50] and DC.js [51] are D3 libraries that provide useful tools for

exactly this kind of scenario; therefore they were used to generate the visualization of this scenario. Their main feature is the use of coordinated views into the data, which in the case of large data sets helps better analyze the outliers and relationships in the dataset. The views attached to this scenario's dataset were two bar graphs, line graph and a table of data, all of which are shown in Figure 26. The first bar graph shows which CPU usage percentages the crashes were mostly found at. The second bar graph shows the amount of bugs found by the different hosts. The line graph depicts the bugs found in the time interval of 24h, and the table shows the selected data in a detailed text form. All of the different graphs can be interacted with a mouse, by using operations such as zooming, dragging and selecting.

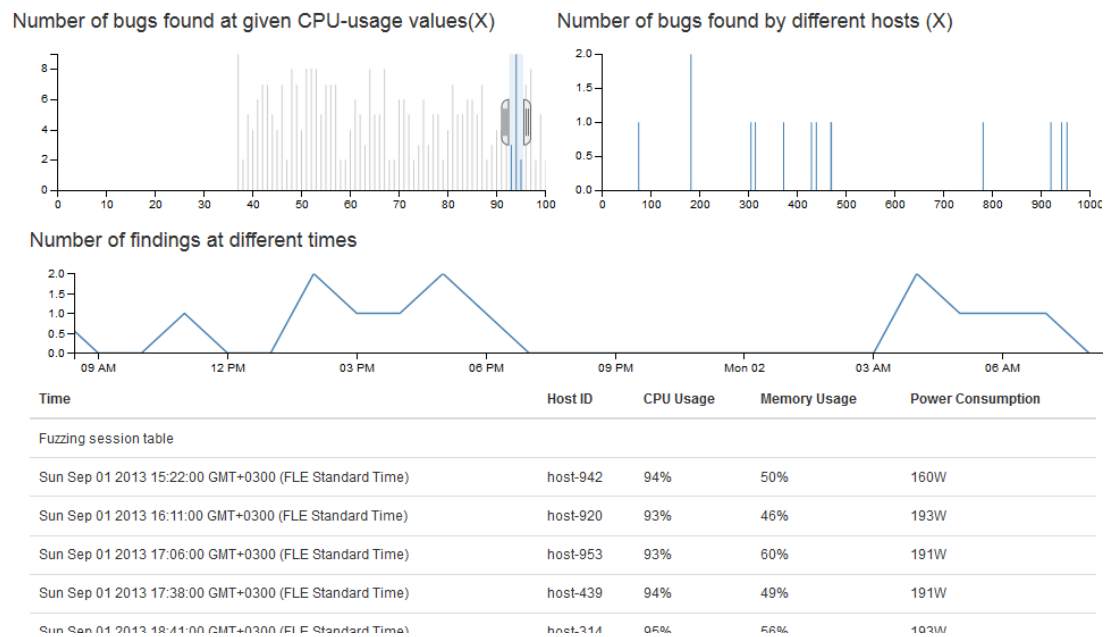


Figure 26. Multiple views on a dataset by using different chart types.

This combination of different graphs shows that interactivity in data visualization and different views into the same material greatly help exploratory analysis of the data. Depending on what kind outliers or patterns are expected to be found, the different graphs can be adjusted in appropriate order and the details can be found in the table for further analysis.

6. DISCUSSION

This chapter restates the main requirements and objectives of this work and analyzes the results of the case studies that were performed. Afterwards, the potential future developments of the software implementation are discussed.

The main objective of the thesis was to examine the usefulness of data visualization in monitoring fuzz testing processes. The aim was to create visualizations on two different scales: single hosts with detailed performance data and simulations of test scenarios with many fuzzers.

6.1. Requirements Revisited

The requirements of the software that were outlined in the design chapter consisted of four distinct parts: data collection, fuzzer implementation, networking and the different visualization scenarios.

The requirements of the data collection were to obtain performance metrics from a single test computer, along with data output from the fuzzer, and pack them conveniently in some format. The data collection was done using the `Collectd` software, which was first configured to send the performance data as CSV files to the monitoring server through its own network plugin. This approach had some problems, for instance, when the CSV files grew large enough the receiving monitoring server would sometimes stop correctly receiving them. Another concern was that from a software design point of view, it seemed more desirable to collect all the different performance data to a single file, which could be unpacked when it arrived to the monitoring server. For these reasons, the single JSON file per host approach was adopted. In the final version of the data collection software, the JSON file contained CPU, memory, bugs found and a Boolean variable indicating whether the test was still running. Because of the time constraints of this thesis, some statistics such as power consumption and network usage had to be dropped.

The main goals in design requirements were to implement a simple ready-made fuzzer that would reliably find crashes in an arbitrary target program. The target program and the overall effectiveness of the fuzzer were unimportant for the scenarios of this thesis. A number of different open source fuzzing frameworks were tried for this scenario, but finally an example Bash script provided by two colleagues was chosen because it was easy to use and modify to fit the required scenario. When it came to the fuzzing targets, new versions of popular web browsers and image viewers were first tried for this scenario, but none of them reliably generated sample crashes with the fuzzer that was used. Finally, an older version of MPlayer [52] that was known to generate crashes with the Radamsa script was chosen as the fuzzing target. Initially, the aim was to use a more complex fuzzing framework, which could provide additional available data about the test process, but due to time limitations to configure them appropriately on the Unix host that was running in the scenario, the simpler custom solution was chosen. In this sense, the software does not fully meet the requirements outlined in the design phase. If more data would have been easily obtained from the fuzzer, more visualizations, such as code coverage, could have been created.

The requirements for the network module of the software were initially to just host a webserver with a ready-made framework such as Apache, and to use a Python

script to obtain the collected data through TCP (Transmission Control Protocol) connections. This approach was soon abandoned in favor of using a custom hybrid application build in Node.js. The benefits were that a single Node.js application could host both the web page content and receive data through its WebSockets implementation. The custom application that was written met both of the requirements, although later examination of the software revealed that if the number of host connections would increase dramatically, some kind of more efficient data streaming approach would probably have to be implemented.

The visualization part of the software had two main requirements. Firstly, it had to be able to visualize in real-time what occurs in the performance measurements at different parts of the fuzzing process. Secondly, it had to be able to visualize a simulations of a larger number of test hosts in such a way that it would be possible to analyze the benefits and drawbacks of different visualization techniques in real world scenarios. The final visualizations had three graphs for single hosts and three graphs for different simulation scenarios, therefore the initial requirements were met. Ideally, more visualizations showcasing different visualization techniques would have been created, and the simulations themselves would have contained an even larger amount of simulated test hosts.

Some difficulties were encountered when trying to fit lots of data into 2D visualizations. For example, Figure 27 shows a circle-packing visualization that initially had real-time animations, where the bubbles were resized dynamically according to the input data.

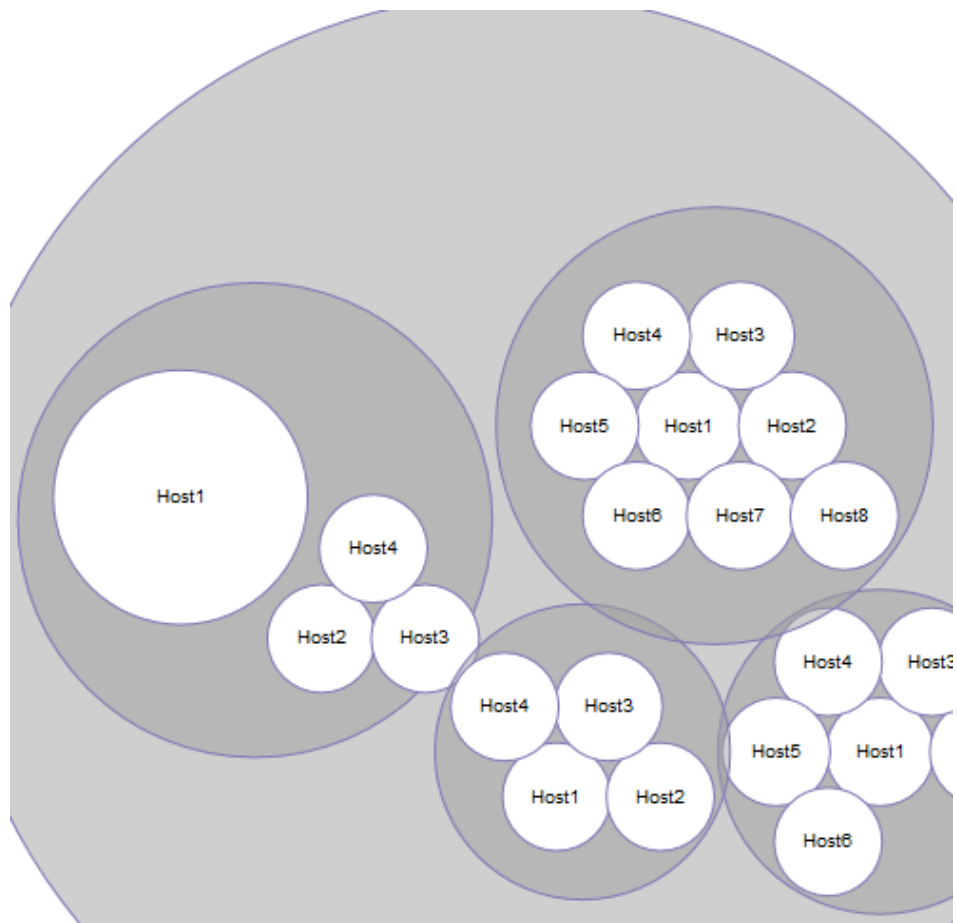


Figure 27. Discarded animated circle-packing visualization example.

. As the number of hosts and their datasets were increased further, the clarity of the visualization started to decrease. Additionally, this kind of visualization could only track one value at a time without cluttering the graph with too many sub-circles. To represent, for example both CPU and memory, two such charts may have been needed.

After discarding the animated circle-packing visualization chart, different techniques were considered on how to represent more data in 2D charts while maintaining clarity of the visualization. Subsequently, three different visualization methods were examined and implemented that could potentially solve the problem. Firstly, interactivity was implemented to the circle-packing visualization, which enabled the user to examine the specific parts of the visualization that were of special interest. This approach enabled the packing of more data into the single visualization while still maintaining clarity and usefulness in data exploration.

The second useful technique was using horizon graphs that pack the data efficiently vertically. In horizon graphs, the intensities of colors of a traditional line graph represent the different values, and the negative values are drawn on the positive axis. This allowed the placing of 20 such graphs parallel to each other on a single web page and therefore increased the data density of the visualization.

The final useful visualization technique was using multiple coordinated views into the data. In the simulation, there were multiple different chart types that gave different views on the dataset, and all of the charts were interactive and usable with mouse.

In conclusion, the two points considered at the beginning, that is, how to represent hierarchy and large multivariate datasets, proved to be useful in this system monitoring scenario. In addition, two new points to consider appeared during the test scenario design process. Firstly, it had to be considered how to increase the data density of the visualization, such as in the case of fitting over 20 charts in to the same view. The use of horizon graphs proved to be very beneficial in this case. Secondly, it was learned, that the use interactivity in the visualization can help when trying to represent large datasets, especially in the case of exploratory visualization. In the case studies where interactivity with the mouse was added, such as zooming and selecting, the clarity of the visualization did not seem to suffer regardless of large datasets.

6.2. Possible Future Developments

Although the final version of the software demonstrated that there is value in using visualization in software testing, there are many areas in which the software could be further developed. Firstly, more performance measurements could be collected from the remote hosts, such as disk utilization, power consumption and network traffic. Additionally, implementing an advanced fuzzing framework that provides more information about the test process could lead to even more interesting visualizations. For example, coordinated views could be provided into power consumption, fuzzer efficiency and money used on testing to monitor real world applications of fuzzers.

The monitoring system could also be developed to have a type of plug-in system, where a new remote host that is running some fuzzer can easily be added to the monitoring system. Ideally, the host could be added to some specified group of fuzzers and set to monitor only certain measurements.

The use of 3D visualizations could also be explored. New technologies would have to be adopted instead of D3.js, but the camera functions of 3D environments could allow much more information to be represented by a single visualization graphic.

7. SUMMARY

In this thesis, the usefulness of data visualization was studied in the context of monitoring fuzz testing processes. A number of different data visualization techniques were presented and examined, in order to select those which could have benefit in the specified testing scenarios. The selected visualization methods were implemented in a software framework, which consisted of a monitoring server and a remote client that were connected via network.

The client remote host was implemented to run a performance data collector and a fuzzer, which was automatized to find crashes in a media player. The remote host was also used to send the collected test data to the main monitoring server over the network. The monitoring server was developed to receive, parse and visualize the test data it received from the remote host. The client-server pair of systems was then used to run a number of case studies, which consisted of visualizing performance data and fuzzing data from a single network host, as well as creating simulations of a large number of fuzzers.

The results of the case studies showed that using data visualization in test case and process monitoring can be beneficial, and that it can potentially provide relevant information quickly to the viewer. Additionally, the results suggested that when creating larger visualizations consisting of many hosts and measurements, the techniques to increase data density and interactivity were both very useful.

8. REFERENCES

- [1] National Vulnerability Database. (accessed 22.8.2013), National Institute of Standards and Technology. URL: <http://nvd.nist.gov>.
- [2] Exploit in Skyrim. (accessed 16.9.2013), Joe's Security Blog. URL: <http://www.gironsec.com/blog/2013/05/exploit-in-skyrim>.
- [3] Mahmood R, Eshafani, N, Kacem, T, Mirzaei N, Malek S & Stavrou A. (2012) A whitebox approach for automated security testing of Android applications on the cloud. Proc. 2012 7th International Workshop on Automation of Software Test (AST), 22-28.
- [4] Icinga: Open Source Monitoring. (accessed 15.11.2013), Icinga: Open Source Monitoring. URL: <https://www.icinga.org/>.
- [5] Sutton M, Greene A & Amini P. (2007) Fuzzing: Brute Force Vulnerability Discovery. Virginia, Addison-Wesley Professional.
- [6] Oehlert P. (2005) Violating Assumptions with Fuzzing. IEEE Security & Privacy 3(2): 58-62.
- [7] Takanen A, DeMott J & Miller C. (2008) Fuzzing for Software Security Testing and Quality Assurance. Norwood, MA, Artech House.
- [8] Schieferdecker I. (2012) Model-Based Fuzz Testing. Proc. 2012 IEEE Fifth International Conference on Software Testing, Verification and Validation (ICST), 814-814.
- [9] Miller BP, Fredriksen L & So B. (1990) An Empirical Study of the Reliability of UNIX Utilities. Commun ACM 33(12): 32-44.
- [10] Dingning Yang, Yuqing Zhang & Qixu Liu. (2012) BlendFuzz: A Model-Based Framework for Fuzz Testing Programs with Grammatical Inputs. Proc. 2012 IEEE 11th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 1070-1076.
- [11] Bekrar S, Bekrar C, Groz R & Mounier L. (2011) Finding Software Vulnerabilities by Smart Fuzzing. Proc. 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation (ICST), 427-430.
- [12] Rosenberg J. (1996) How Debuggers Work: Algorithms, Data Structures, and Architecture. New York City, NY, John Wiley & Sons, Inc.
- [13] Dingning Yang, Yuqing Zhang & Qixu Liu. (2012) BlendFuzz: A Model-Based Framework for Fuzz Testing Programs with Grammatical Inputs. Proc. 2012 IEEE 11th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 1070-1076.

- [14] Tielei Wang, Tao Wei, Guofei Gu & Wei Zou. (2010) TaintScope: A Checksum-Aware Directed Fuzzing Tool for Automatic Software Vulnerability Detection. Proc. 2010 IEEE Symposium on Security and Privacy (SP), 497-512.
- [15] Sparks S, Embleton S, Cunningham R & Zou C. (2007) Automated Vulnerability Analysis: Leveraging Control Flow for Evolutionary Input Crafting. Proc. 2007 Twenty-Third Annual Computer Security Applications Conference (ACSAC), 477-486.
- [16] Cadar C, Dunbar D & Engler D (2008) KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. Proc. 2008 8th USENIX Symposium on Operating Systems Design and Implementation, 209-224.
- [17] McNally R, Yiu K, Grove D & Gerhardy D (2012) Fuzzing: The State of the Art. URL: http://www.dsto.defence.gov.au/publications/scientific_record.php?record=10173 . Accessed 15.11.2013.
- [18] Global Data Storage Calculated at 295 Exabytes. (accessed 21.5.2013), BBC News. URL: <http://www.bbc.co.uk/news/technology-12419672>.
- [19] Yau N. (2011) Visualize This: The FlowingData Guide to Design, Visualization, and Statistics. Indianapolis, IN, Wiley.
- [20] Friendly M. (2008) Milestones in the History of Thematic Cartography, Statistical Graphics, and Data Visualization. Boston, Seeing Science.
- [21] Myatt G.J. & Johnson W.P. (2011) Making Sense of Data III: A Practical Guide to Designing Interactive Data Visualizations. New Jersey, Wiley.
- [22] van Wijk JJ. (2005) The Value of Visualization. Proc. 2005. IEEE VIS 05, 79-86.
- [23] Iliinsky N & Steele J. (2011) Designing Data Visualizations: Representing Informational Relationships. Sebastopol, CA, O'Reilly Media.
- [24] Anscombe F. (1973) Graphs in Statistical Analysis. The American Statistician 27(1): 17-21.
- [25] Fry B. (2008) Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, O'Reilly Media.
- [26] Chen C, Härdle W & Unwin A. (2007) Handbook of Data Visualization. Berlin, Springer-Verlag.
- [27] Milestones in the History of Thematic Cartography, Statistical Graphics, and Data Visualization (accessed 19.9.2013), Friendly M. & Denis J. URL: <http://datavis.ca/milestones/>.

- [28] Tufte E. (2006) Beautiful Evidence. CT, Graphics Press Cheshire.
- [29] Krzywinski M, Schein J, Birol I, Connors J, Gascoyne R, Horsman D, Jones S & Marra M. (2009) Circos: An Information Aesthetic for Comparative Genomics. *Genome Research*, June 18.
- [30] Chord Layout (accessed 15.1.2014), Mike Bostock. URL: <https://github.com/mbostock/d3/wiki/Chord-Layout>.
- [31] Harris R (1999) Information Graphics: A Comprehensive Illustrated Reference. Atlanta, Georgia, Oxford University Press.
- [32] Heer J, Bostock M & Ogievetsky V (2010) A Tour Through the Visualization Zoo. *Queue - Visualization* 8(6): 1-22.
- [33] Wang W, Wang H, Dai G & Wang H. (2006) Visualization of Large Hierarchical Data by Circle Packing. *Proc. 2006 CHI'06 Conference on Human Factors in Computing Systems*, ACM, 517-520.
- [34] Heer J, Kong N & Agrawala M. (2009) Sizing the horizon: the effects of chart size and layering on the graphical perception of time series visualizations. *Proc. 2009 CHI'09 Conference on Human Factors in Computing Systems*, ACM, 1303-1312.
- [35] Apache HTTP Server Project (accessed 15.11.2013), Apache Software Foundation. URL: <http://httpd.apache.org>.
- [36] Django (accessed 15.11.2013), Django Software Foundation. URL: <https://www.djangoproject.com>.
- [37] Node.js (accessed 19.9.2013), Joyent, Inc. URL: <http://nodejs.org>.
- [38] Socket.IO (accessed 15.11.2013), Guillermo Rauch. URL: <http://socket.io>.
- [39] Express – Web Application Framework for Node (accessed 15.11.2013), Vision Media. URL: <http://expressjs.com>.
- [40] WebSocket.org (accessed 15.11.2013), Kaazing Corporation. URL: <http://www.websocket.org>.
- [41] Oline A & Reinert D. (2005) Exploring Three-dimensional Visualization for Intrusion Detection. *Proc. 2005 IEEE Workshop on Visualization for Computer Security (VizSEC)*, 113-120.
- [42] The 20 Best Tools for Data Visualization (accessed 19.9.2013), Creative Bloq. URL: <http://www.creativebloq.com/design-tools/data-visualization-712402>
- [43] D3.js – Data-Driven Documents (accessed 15.11.2013), Michael Bostock. URL: <http://d3js.org>.

- [44] Bostock M, Ogievetsky V & Heer J. (2011) D3 Data-Driven Documents. Proc. 2011 IEEE Transactions on Visualization and Computer Graphics, 17, 12, 2301-2309.
- [45] Bostock M & Heer J. (2009) Protovis: A Graphical Toolkit for Visualization. Proc. 2009 IEEE Transactions on Visualization and Computer Graphics, 15, 6, 1121-1128.
- [46] Munin (accessed 15.11.2013), Munin-Monitoring. URL: <http://munin-monitoring.org>.
- [47] Collectd – The System Statistics Collection Daemon (accessed 15.11.2013), Collectd. URL: <http://collectd.org>.
- [48] Radamsa (accessed 15.11.2013), OUSPG. URL: <https://www.ee.oulu.fi/research/ouspg/Radamsa>.
- [49] Cubism.js (accessed 15.11.2013), Square, Inc. URL: <http://square.github.io/cubism>.
- [50] Crossfilter (accessed 15.11.2013), Square, Inc. URL: <http://square.github.io/crossfilter>.
- [51] Dc.js – Dimensional Charting Javascript Library (accessed 15.11.2013), Nick Zhu. URL: <http://nickqizhu.github.io/dc.js>.
- [52] MPlayer – The Movie Player (accessed 15.11.2013), MPlayer Team. URL: <http://www.mplayerhq.hu>.